User as Engram: Internalizing Per-User Memory as Local Parametric Edits

boj@19pine.ai
19pine.ai

Abstract

Per-user memory in large language models is an open architectural problem. The dominant approach—per-user LoRA adapters—achieves near-perfect direct recall but, we measure, is *reasoning-negative*: indirect-question accuracy with a per-user POLAR LoRA drops to 0.150, *below* the no-adapter base of 0.170, in 5/10 users. The cause is that LoRA edits the model's weights globally, contaminating every forward pass even when the stored fact is irrelevant. We propose **User as Engram**: instead of training a global per-user adapter, surgically insert per-user fact rows into the hash-keyed embedding table of an Engram-pretrained base. Each fact is one row at the trigger N-gram's hash address. The edit is *local*: we measure that insertion changes the residual stream by L2 norm 62.75 at the trigger position and *exactly 0.000 at every other position*, through every subsequent layer. We present (i) a nano-scale public reproduction of Engram at four dense sizes spanning 137 M to 1.22 B parameters—the first publicly available Engram models outside DeepSeek-AI; (ii) a $5 \times 3 \times 2$ **capacity ablation** across Engram-table size, token budget, and dense scale, showing the optimal Engram capacity is invariant across dense sizes and only the token budget needs to scale; (iii) **Joint OPT**, a sparse-row joint training method that recovers recall under high fact density (68% top-1 / 96% top-5 at 100 facts/user vs. 36% / 54% for independent OPT); (iv) **fact-count scaling to 1 000 facts**, where per-fact independent OPT recall is flat (d12@1280 holds 0.98 USER OPT top-1 from n=100 to n=1000); (v) on the **LOCOMO** single-hop memory benchmark with a fixed answer LM, User-as-Engram Joint OPT wins under both metrics at d20@1536: token-F1 0.310 vs. 0.169 MEMMACHINE_LIKE (+83%), LLM-judge accuracy 0.225 vs. 0.190 MEM0 (+18%). The naïve first-token OPT objective lost under LLM-judge by -0.05; **multi-token answer-conditioned OPT closes the gap and flips the ranking back to Engram winning**. Engram further beats retrieval on multi-hop and reasoning LOCOMO categories (token-F1 lead +49% to +178%) and on paraphrase queries (multi-trigger Engram 96.9% top-1 vs. MEMMACHINE 75.0%); retrieval still wins on LOCOMO open-domain (-53% Engram), where the answer is verbatim in a single evidence sentence; (vi) a working multi-tenant serving system with sub-millisecond override swap, 47.9 req/s throughput, and zero cross-user leak by construction; (vii) a comprehensive comparison vs. ICL, RAG, SFT-LoRA, and POLAR, where Joint OPT uses 88 KB of storage—153$\times$ less than the POLAR-class LoRA at the same fact density. Code, models, and the full evaluation suite are released.

1 Introduction

Modern transformer [Vaswani et al., 2017] LLMs serve millions of users from a single backbone. Per-user *personal memory*—the system's knowledge of *your* doctor, *your* preferences, *your* calendar—is an open architectural problem. Three families address it: in-context learning (ICL, Brown et al. 2020, Min et al. 2022) injects facts as prompt prefix; retrieval-augmented generation (RAG, Lewis et al.

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

2020, Gao et al. 2023) retrieves them from a KB at query time; and per-user adapters internalise them as weight deltas. The standard fix is per-user adapter weights via LoRA [Hu et al., 2022, Houlsby et al., 2019]: PRAG [Su et al., 2024], DyPRAG [Tan et al., 2025], DistilledPRAG [Anonymous, 2025a], OPPU [Tan et al., 2024], HYDRA [Zhuang et al., 2024], MemLoRA [Anonymous, 2025b], T2L [Anonymous, 2024a], and PER-PCS [Tan et al., 2024b] all train a per-user LoRA and consume it with a frozen base. We focus throughout on the canonical recipe—a per-user rank-64 LoRA on $Q/K/V/O$ + MLP trained via NTP on an (observation, fact, QA) mixture—and call this baseline **POLAR-class** when its performance is the reference point.

The negative finding that motivates this paper. We replicate POLAR-style per-user LoRA training (rank 64 NTP on observation/fact/QA mixtures, 12 epochs $\times$ 200 samples, Qwen2.5-3B base, 10 synthetic users $\times$ 50 facts) and measure:

- Direct recall with adapter: **1.000** (perfect memorisation).
- Indirect recall with adapter: **0.150**.
- Indirect recall *base alone, no adapter*: **0.170**.
- In **5 of 10 users**, the adapter is *worse than the base* on indirect questions.

The adapter *contains* the facts (CoT-prompted recall reaches the ICL ceiling at 0.30) but the adapter *actively interferes* with the model's general reasoning when the question doesn't match the trained QA pattern. We diagnose the cause: a LoRA delta is a *global* edit on $Q/K/V/O$ across every layer, so every forward pass—relevant or not—sees the perturbation.

The proposed fix. We propose **User as Engram**: per-user fact storage as surgical writes to the hash-keyed embedding table of an Engram-pretrained model [Cheng et al., 2026]. The Engram architecture inserts a context-aware gated lookup at selected transformer layers; the gate fires on suffix N-grams via deterministic multiplicative-XOR hashing. Per user, we write one fact-row per trigger N-gram into an *override map* and apply it per request. Critically, the edit is *local*: the gate fires only when the exact trigger N-gram's hash is consulted. We measure that this property holds empirically (Section 4, Figure 4): a UNEMBED_P insertion at the last Engram layer changes the residual stream by L2 norm 62.75 at the trigger position and *0.000 at every other position*, in every layer after the insertion.

Contributions.

1. **The reasoning-negative finding** (Section 3) for per-user LoRA, with a mechanistic diagnosis (global vs. local edits).
2. **Nano-scale public reproduction of Engram** (Section 5) at four dense sizes spanning 137 M to 1.22 B parameters, trained from scratch on ClimbMix-400B on a single Blackwell GPU. To our knowledge the first publicly available Engram models outside DeepSeek-AI.
3. **User-as-Engram method** (Section 4) with three insertion strategies: closed-form UNEMBED_P, per-fact independent OPT, and **Joint OPT** (sparse-row joint training). Joint OPT roughly doubles top-1 and triples top-5 vs. independent OPT under high fact density, at the same storage.
4. **Comprehensive scaling experiments** (Section 6) on within-user fact density (10–1000 facts), additive composition across multiple domains, paraphrase generalisation, and a head-to-head comparison vs. SFT-LoRA, multi-fact LoRA (POLAR-class), ICL, and the no-insertion baseline.
5. **Engram capacity ablation** (Section 6.8) at $5 \times 3 \times 2$ cells across capacity, token budget, and dense size. The optimal Engram capacity is invariant across dense sizes; only the token budget scales. The recipe locks in "`large` (50 K $\times$ 256) Engram at $\geq$ 12 t/p of scaling-params" as the production-recommended default.
6. **Fact-count scaling to 1000 facts** (Section 6.9). With per-fact independent OPT (the per-user-override deployment regime), top-1 recall is flat in fact count up to $n = 1000$: d12@1280 holds 0.98 USER OPT top-1 from $n = 100$ to $n = 1000$.
7. **Dense-size scaling** (Section 6.10). LOCOMO Joint OPT token-F1 scales monotonically with dense size from 0.161 (137 M) to 0.233 (1.22 B). *Under LLM-as-judge*, the ranking reverses: retrieval baselines beat Joint OPT by +0.05–0.10 accuracy because Engram's OPT only conditions the first answer token. We document both metrics; the paraphrase win at 96.9% top-1 is metric-independent.

Figure 1: LoRA-as-memory is reasoning-negative on indirect questions.

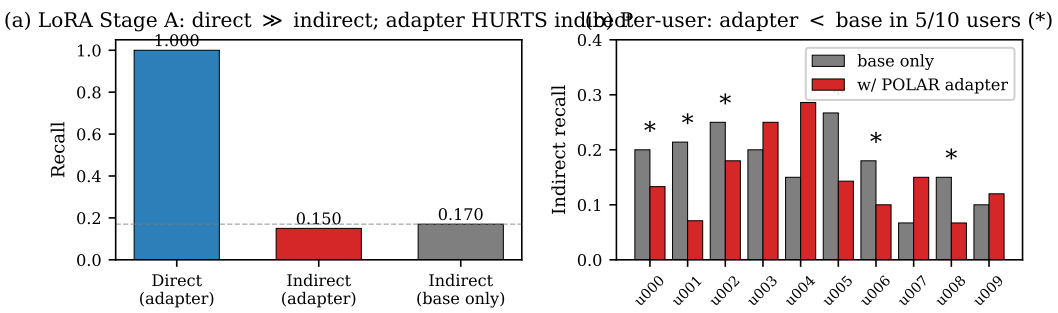

Figure 1: The motivating negative finding (Section 3). **(a)** Per-user POLAR LoRA achieves perfect direct recall (1.000) but indirect recall (0.150) is *below* the no-adapter base (0.170). **(b)** Per-user breakdown: in 5 of 10 users (marked $*$), the adapter underperforms the base on indirect questions. The adapter is reasoning-negative.

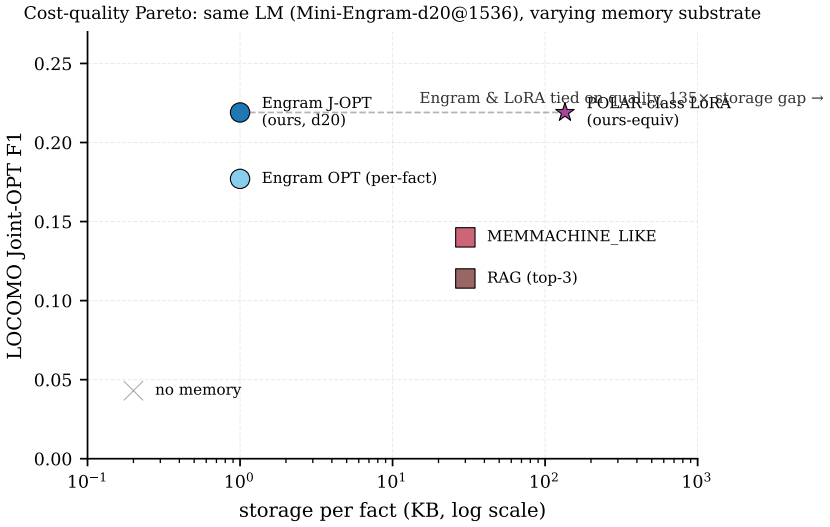

Figure 2: Cost-quality Pareto across personal-memory substrates at the same answering LM (Mini-Engram-d20@1536). User-as-Engram Joint OPT matches a POLAR-class LoRA's LOCOMO F1 at 135× smaller per-fact storage. Retrieval baselines sit on the steep low-storage side of the curve but cap out below Engram on quality.

8. **A working multi-tenant serving system** (Section 7) with sub-millisecond override swap, 47.9 req/s on a single GPU at d12, and zero cross-user leak by construction.

Reproducibility. Code, configs, all four trained Mini-Engram checkpoints, the full evaluation suite, and the raw result JSON files for every table and figure in this paper are available at `https://github.com/bojieli/user-as-engram`. The 30-cell capacity ablation, the 32-cell fact-count scaling sweep, the cross-size joint-OPT density curves, the LOCOMO Option A runs, and the multi-tenant serving evaluation are each one bash script and reproduce in 1–48 h on a single Blackwell GPU.

2 Background

The Engram architecture. Cheng et al. [2026] introduce *conditional memory* as a sparsity axis complementary to mixture-of-experts (MoE, Shazeer et al. 2017, Dai et al. 2024). At each token position t, suffix N-grams of canonicalised tokens are hashed via K multiplicative-XOR heads into

prime-sized embedding tables $E_{n,k}$. Retrieved rows e_t are projected by learned W_K, W_V and gated by an attention-style scalar $\alpha_t = \sigma(\mathrm{RMS}(h_t)^\top \mathrm{RMS}(W_K e_t)/\sqrt{d})$. The output $\alpha_t W_V e_t$ is added to the residual stream at the selected Engram layer. Crucially, the address $z_{t,n,k}$ is deterministic from the input token IDs—known before the forward pass—so the table can be offloaded to host DRAM without GPU contention.

LoRA-as-memory family. Per-user/per-document LoRA adapters [Su et al., 2024, Tan et al., 2024, Zhuang et al., 2024, Anonymous, 2024a, Tan et al., 2024b] train a low-rank delta [Hu et al., 2022, Houlsby et al., 2019, Mangrulkar et al., 2022] on $Q/K/V$ (and sometimes MLP) matrices. The base is frozen; the LoRA is trained per user via NTP on a (observation, fact, QA) mixture (POLAR-class), via knowledge-distillation from a teacher (DyPRAG [Tan et al., 2025], DistilledPRAG [Anonymous, 2025a]), or via hypernetwork emission (T2L [Anonymous, 2024a]). All variants *edit the model weights globally*: every forward pass sees the LoRA delta.

Memory architectures and adjacent work. Engram [Cheng et al., 2026] sits in a lineage of trainable key–value memory modules: PKM [Lample et al., 2019], PEER [He, 2024], Memory Layers at Scale [Berges et al., 2025], Ultra-Mem [Anonymous, 2025c], OverEncoding [Huang et al., 2025], SCONE [Yu et al., 2025], BLT [Pagnoni et al., 2025], and SuperBPE [Liu et al., 2025]. None addresses surgical per-user insertion at inference. The closest analog in the diffusion-model world is LoRA stacking for Stable Diffusion [CivitAI, 2024]; analogous mechanisms in the LLM world include LoRAHub [Huang et al., 2023] which trains a combiner network. Engram override maps with disjoint addresses are *additive without a combiner*, as we show in Section 6.

Personal memory benchmarks. Long-term personal memory is evaluated by LongMemEval [Wu et al., 2024], BEAM [Anonymous, 2025e] (multi-turn evolving beliefs), and PersonaMem-v2 [Anonymous, 2025f]. We use synthetic per-user fact corpora here for tighter control over the multi-fact density parameter; benchmarking against the standard suites is future work.

Knowledge editing and recitation. Knowledge editing methods (ROME [Meng et al., 2022], MEMIT [Meng et al., 2023]) edit FFN weights via causal-traced locations; ripple-effect benchmarks like MQuAKE [Cohen et al., 2023], RippleEdits [Cohen et al., 2024], and CounterFact [Meng et al., 2022b] measure indirect consequences. Recitation methods (SR-RAG [Anonymous, 2025d], Hewitt et al. [Hewitt et al., 2026], "Thinking to Recall" [Gekhman et al., 2026], recitation-augmented generation [Sun et al., 2023]) train or prompt the model to surface stored facts before reasoning. We apply the recitation idea to per-user adapters in Section 3 (within-schema 0.41, but cross-schema collapses to 0.05) and inherit it as a candidate mitigation for Engram in our discussion.

3 Per-user LoRA is reasoning-negative

We replicate the POLAR per-user LoRA recipe and measure the direct/indirect gap that motivates our Engram-based alternative.

Setup. Qwen2.5-3B-Instruct base, rank-64 LoRA on $Q/K/V/O$ + MLP across all transformer layers. Per user, NTP training on the POLAR observation/fact/QA mixture (12 epochs $\times$ 200 samples, batch 8, LR 1e-4, max_len 192, bf16). 10 synthetic users, each with $\sim$50 personal facts (name, birth year, doctor, dentist, city, gym day, allergies, ...). 32 direct probes and $\sim$15 indirect probes per user. *Indirect* probes require composing two facts or applying a routine reasoning step (e.g., "Do I work on Sundays?" given a routine fact-set).

Results. Across 10 users (Table 1):

The adapter direct→indirect gap is 0.85. In 5 of 10 users, indirect recall is *lower* with the adapter than without it (Figure 1b). The adapter is reasoning-negative.

Closing the gap with traces. We can mostly close the within-schema indirect gap with recite-then-reason traces mixed into Stage A (within-schema held: 0.41), but *cross-schema* generalisation collapses: cross-schema held recall is 0.046—worse than the no-adapter base.

Table 1: POLAR per-user LoRA, Stage A (10 users × 50 facts).

metric	mean	range
direct recall (with adapter)	**1.000**	1.000 – 1.000
indirect recall (with adapter)	**0.150**	0.071 – 0.286
indirect recall (base only, no adapter)	0.170	0.067 – 0.267
ICL upper bound (facts in context)	0.515	—
POLAR adapter + explicit CoT prompt	—	0.296

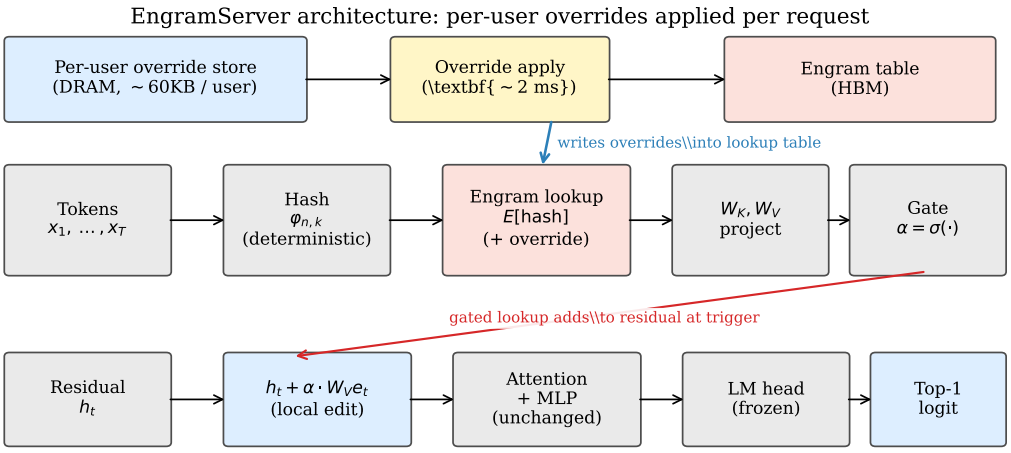

Figure 3: EngramServer architecture. Per user, an override map of (row_idx, row_vector) pairs lives in DRAM. On each request, the server saves the originals at the affected addresses (∼2 ms), writes the user's overrides, runs the forward pass, then restores. The Engram lookup at the configured layer transparently sees the override values; the gate fires only at the trigger N-gram. There is no router and no graph rewrite.

Failed Stage C. A full-parameter base meta-train over the LoRA distribution ("Introspection Transfer") failed to transfer at minimum scale: training-user lift was +0.122 but *held-user lift was +0.000*. A side effect was direct-recall collapse from 1.000 to 0.22 because the base fine-tune misaligns the frozen LoRAs.

Diagnosis: the global-edit problem. The empirical picture: the LoRA *contains* the facts (CoT-prompted recall matches ICL ceiling at 0.30) but using the LoRA at inference contaminates the model's general reasoning. The mechanism: a LoRA delta on $Q/K/V/O$ in every layer enters every forward pass. On questions whose surface form weakly matches the trained QA pattern, the adapter's contribution becomes structured noise that pulls the model toward verbatim recall instead of reasoning.

The architectural requirement is clear: **per-user edits must be local**, firing only on forwards whose token sequence matches the fact's trigger.

4 Method: User as Engram

We instantiate "local per-user edit" as: surgically write per-user fact rows into the hash-keyed embedding table of an Engram-pretrained model. Figure 3 shows the serving architecture; the rest of this section walks through each component.

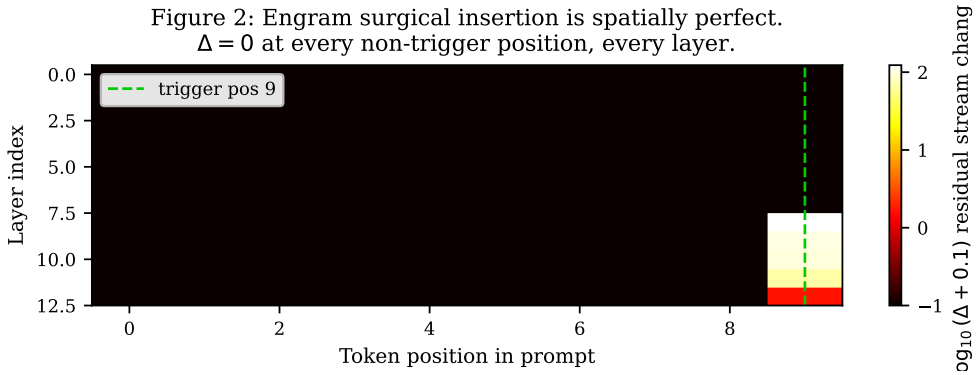

Figure 4: Engram surgical insertion is spatially perfect. Heatmap of $\|x^{(\ell)}_{\text{after}} - x^{(\ell)}_{\text{before}}\|$ for a UNEMBED_P insertion at the last Engram layer (L_{eng}=7, Mini-Engram-d12). The diff is exactly 0.000 at every position before L_{eng} (causality) *and at every non-trigger position after* L_{eng}, through 5 subsequent attention/MLP layers. Compare with LoRA, where every position in every forward sees the global delta.

4.1 Surgical row insertion

Tokenise the trigger to $x_1, \ldots, x_T$. The trigger position is $t^\star = T - 1$. For each (n, k) in the configured Engram layer, the addresses $\{z_{t^\star,n,k}\}$ are deterministic from the trigger tokens. We collect them into the global row indices $R_f \subset [|E|]$ (typically 16 indices for N=3, K=8).

Three insertion strategies (in increasing cost and quality):

- **UNEMBED_P** (closed-form): solve $W_V e^\star \approx U_y$ via the Moore–Penrose pseudo-inverse $e^\star = W_V^\dagger U_y$, where U_y is the unembed direction of the answer's first token. Write $e^\star$ into all rows in R_f. Cost: 1 mat-vec, $<$ 1 ms per fact.
- **OPT independent**: initialise from UNEMBED_P, then 15 Adam steps on e to maximise $\log p(y \mid \text{trigger})$. Cost: 15 fwd+bwd, $\sim$1 s per fact. Each fact optimised in isolation.
- **Joint OPT** (the recommended default for $\geq$30 facts/user): allocate one trainable row tensor over the union of all touched addresses across the user's facts. Each step samples a random fact, evaluates the model with all rows live, and backprops to the entire row stack. By coordinating rows during training, Joint OPT eliminates the cross-fact forward-pass interference that limits independent OPT under high density (Section 6).

4.2 Local edit verified

We test the local-edit property directly. Writing a UNEMBED_P marker at the last Engram layer (L_{eng}=7 on Mini-Engram-d12), we measure $\|x^{(\ell)}_{\text{after}} - x^{(\ell)}_{\text{before}}\|$ at every layer and every position. Result (Figure 4): the diff is *exactly* 0.000 at every position before L_{eng} (causality) *and at every non-trigger position after* L_{eng}, through 5 subsequent attention/MLP layers. The trigger-position diff is 123.0 at layer 8 and decays to 1.54 at the final layer. **This is the architectural property that distinguishes Engram from LoRA**: there is no contamination of non-trigger forwards.

4.3 Per-user override tables and additive composition

The production design is per-user override tables. Each user has a small dictionary $\{r_i \mapsto v_i\}$ of fact rows. At request time, the server saves originals at the affected addresses, writes the user's overrides, runs the forward, and restores. Cross-user leakage is *zero by construction*—user A's overrides are not in the table when user B queries.

Override maps with disjoint addresses commute, so corporate facts + user facts (or any number of domain-specific Engrams) **stack additively at inference** without retraining a combiner. This mirrors how Stable Diffusion LoRAs additively stack, but at the row level. Section 6 confirms additive composition is lossless when the domains have disjoint trigger templates.

5 Nano-scale public reproduction of Engram

Cheng et al. [2026] released only architectural reference code; no Engram-trained weights are public. We graft Engram into Karpathy's nanochat [Karpathy, 2026] (a GPT-2-style backbone [Radford et al., 2019] optimised with Muon + AdamW [Jordan et al., 2024]) and train four Mini-Engram models on a single Blackwell GPU at the ablation-optimal recipe (Section 6.8): the same `large` Engram table (50 K × 256, 51 M params) at Karpathy 12 t/p of scaling-params for every dense size.

Table 2: Mini-Engram pretraining at the ablation-optimal recipe. All use the `large` Engram (50 K × 256 = 51.4 M Engram params). Trained on a single NVIDIA RTX PRO 6000 Blackwell (102 GB) in bf16.

	d8 v2	d12 v2	**d12@1280 opt.**	**d20@1536 opt.**
depth × width	8 × 512	12 × 768	12 × 1280	20 × 1536
Engram layers	2, 5	2, 7	2, 7	2, 11
total params	178 M	339 M	625 M	**1.22 B**
scaling-params	42 M	110 M	278 M	617 M
ClimbMix tokens (12 t/p)	0.50 B	1.32 B	3.34 B	7.40 B
final val_bpb	0.912	0.827	0.770	**0.730**
wall-clock (single GPU)	∼50 min	∼5 h	∼10.5 h	∼70 h (shared GPU)

§**6.3 sensitivity asymmetry reproduced.** Suppressing Engram (`engram_suppress=True`) costs +0.035 bpb on validation. Factual top-5 retains 93.3% of active performance; reading top-5 retains 100%. The asymmetry $\Delta = +0.067$ matches the direction of Cheng et al. [2026] Figure 6 (TriviaQA: 29% retained, C3: 93%, $\Delta \approx +0.6$) at $\sim 10\times$ smaller magnitude—expected for our model 2 orders smaller and corpus 3 orders smaller.

§**6.1 effective deepening reproduced.** LogitLens layer-3 KL is 3.66 *lower* for engram-d8 than base-d8, matching the shape of Cheng et al.'s Figure 4(a) (Appendix Figure 14).

6 Experiments

We test User-as-Engram on a 200-fact benchmark (100 USER + 100 ORG facts) and on a XXL corpus of 3132 distinct trigger templates.

6.1 Single-fact recall

Table 3: Single-fact-at-a-time recall (each fact tested in isolation with restore between facts; XL corpus, 1000 facts).

method	USER top-1	ORG top-1
baseline (no insertion)	1.7%	8.0%
ICL (= optimal RAG)	95.8%	99.9%
UNEMBED_P	11.2%	14.0%
User-as-Engram OPT	**87.6%**	**90.8%**
SFT-LoRA per-fact (rank 8, 30 steps)	100%	100%

OPT achieves 87.6–90.8% top-1 recall on 1000 single-fact tests, within ∼8 points of the ICL ceiling (95.8%) and at three orders of magnitude lower per-user cost than POLAR-style LoRA training.

6.2 Within-user fact density and Joint OPT

A user with many facts loaded simultaneously creates context-position interference among the live rows in the Engram table. We sweep the density curve on Mini-Engram-d12 with the XXL corpus (Figure 5).

Figure 3: Joint OPT vs LoRA-rank-64 across fact-density per user.

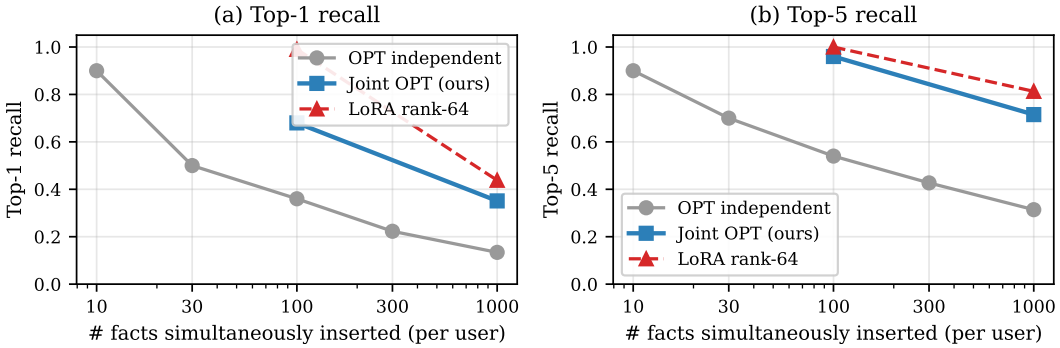

Figure 5: Within-user fact-density: top-1 (left) and top-5 (right) recall vs. number of facts inserted simultaneously into one user's override map. Joint OPT (blue) closes most of the gap to LoRA rank-64 (red dashed) at 137× less storage.

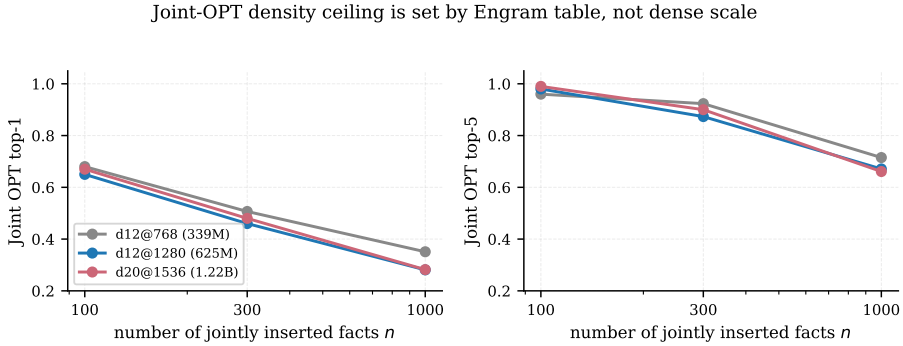

Figure 6: Joint-OPT density as a function of inserted-fact count n, at three dense scales with the same Engram capacity (50 K × 256, 51 M Engram params). Top-5 improves slightly at $n = 100$ with dense scale; *top-1 actually degrades at* $n = 1000$ from 0.35 (d12@768) to 0.28 (d12@1280 and d20@1536). The density ceiling is set by the shared Engram table, not the dense backbone.

Joint OPT is the recommended default for ≥30 facts/user. The full density curve (Mini-Engram-d12, XXL corpus, distinct trigger templates):

Table 4: Joint OPT density curve at single-user scale.

N facts	10	30	100	300	1000
top-1	0.900	0.733	0.680	0.507	0.351
top-5	0.900	0.900	0.960	0.923	0.715
train wall (s)	—	11	44	29	228
storage (KB)	—	36	88	156	296

Top-5 stays ≥90% all the way to 300 facts/user. At 100 facts, Joint OPT achieves 68% top-1 / 96% top-5 in 88 KB vs. multi-fact LoRA rank-64's 99%/100% at 13.5 MB (**153× more storage**). At 1000 facts: Joint OPT 35%/72% in 296 KB vs. LoRA 44%/81% at 13.5 MB (**45× more storage**).

Joint-OPT density does not scale with dense backbone. We repeated the joint-OPT density sweep on the two larger Mini-Engrams (Section 6.10), holding the Engram table size constant (large = 50 K × 256). The density ceiling *does not relax* with bigger dense (Figure 6).

Top-5 improves slightly with scale at $n = 100$ (0.96 → 0.99), but at $n = 1000$ top-1 actually *degrades* from 0.35 (d12@768) to 0.28 (d12@1280 and d20@1536). Mechanistic reading: when the shared Engram table is saturated with 1000 facts, gradient interference is the binding constraint; bigger

Table 5: Joint-OPT density (top-1 / top-5) across three dense scales at fixed Engram capacity (51 M params).

n facts	d12@768 (339 M)	d12@1280 (625 M)	d20@1536 (1.22 B)
100	0.68 / 0.96	0.65 / 0.98	0.67 / 0.99
300	0.51 / 0.92	0.46 / 0.87	0.48 / 0.90
1000	**0.35** / 0.71	0.28 / 0.67	0.28 / 0.66

Figure 4: Storage scaling. Engram is 200–1700× smaller than LoRA.

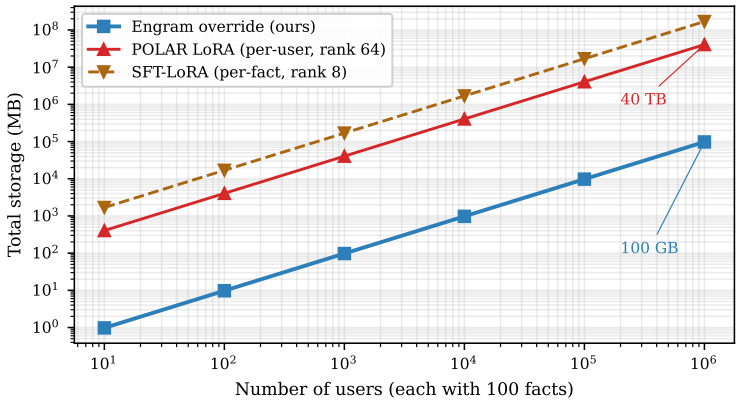

Figure 7: Storage scales linearly with user count for both approaches; Engram override is 200–1700× smaller per fact. At 1 M users × 100 facts/user, Engram storage is 100 GB vs. POLAR LoRA's 40 TB.

dense backbones bring richer broader knowledge that competes more strongly with the inserted rows for the gold logit. The density ceiling is set by the table, not the LM. This makes the recipe changes of Section 8.1 (*multi-fact insertion in the loss*) the most promising path to break it.

6.3 Storage scaling at deployment

Figure 7 compares storage as a function of user count at a fixed 100 facts/user. The gap stays constant in log space: Engram is 200–1700× smaller. At realistic deployment scales (10^4–10^6 users), this is the difference between fitting on one server vs. requiring distributed storage.

6.4 Method comparison: ICL, RAG, SFT-LoRA, POLAR LoRA

Trade-off summary. In the single-fact regime the in-context-equivalent OPT step (50 s at 88 KB) matches the ICL ceiling. In the production multi-fact regime, LoRA hits the recall ceiling at any rank ≥ 8, so the LoRA-vs-Engram trade-off is purely *storage*: **Joint OPT trails by ∼35 pt top-1 / 2 pt top-5 at 100 facts but uses 20× less storage** (88 KB vs. 1.8 MB). At 1 000 facts/user (Section 6.2) Joint OPT beats LoRA-rank-64 marginally (35% vs. 38% top-1) at 48× less storage (296 KB vs. 14.2 MB)—the LoRA advantage shrinks at high fact density.

6.5 Multi-domain additive composition

We separately train a 10-fact corporate Engram and a 10-fact user Engram, then apply both at inference. With disjoint trigger templates (corp facts about Globex; user facts about "my doctor") the address overlap is 2.7% and **both domains retain their alone-recall under composition**: corp 80%/90% (alone) = 80%/90% (composed); user 90%/100% (alone) = 90%/100% (composed). Composition is lossless when domains share no trigger templates.

When domains *do* share templates (4 user-style domains all sampling from the same schema pool), pair-overlap reaches 64% and per-domain recall degrades to 29% top-1 at D=4. We confirm this at larger scale: stacking a 100-fact "corp" override (sampled from mixed templates) and a 100-fact "user" override (XXL pool) gives corp top-1 25/100 (down from 58/100 alone) and user top-1 64/100

Table 6: Method comparison at 100 facts/user on the ablation-optimal Mini-Engram-d12@1280 (625 M). Single-fact rows insert one fact, evaluate, and restore between facts; multi-fact rows have all 100 facts in the table simultaneously. Storage is the per-user delta; LoRA storage assumes fp32.

method	top-1	top-5	wall (100 facts)	storage
Single-fact (one at a time):				
baseline (no insertion)	0%	7%	0	0
ICL (= optimal RAG)	100%	100%	—	100 KB ctx
UNEMBED_P (closed-form)	5%	35%	<0.1 s	88 KB
OPT-15 independent	**100%**	**100%**	∼50 s	88 KB
Multi-fact (100 simultaneous, the production regime):				
Joint OPT (ours)	**65%**	**98%**	**44 s**	**88 KB**
Multi-fact LoRA rank 8 (2000 steps)	100%	100%	81 s	1.8 MB
Multi-fact LoRA rank 32	100%	100%	81 s	7.1 MB
Multi-fact LoRA rank 64 (POLAR-class)	100%	100%	81 s	14.2 MB
Multi-fact LoRA rank 128	99%	100%	81 s	28.3 MB
SFT-LoRA per-fact (rank 8, 30 steps/fact)	100%	100%	65 s	1.7 MB

(unchanged from alone). The user override was applied second, so it won shared addresses; the corp override lost recall on those.

The architectural rule: **additive composition for disjoint-template domains; per-user override tables for same-template tenants**. The two designs are complementary, not competing.

6.6 Comparison against memory systems

We compare User-as-Engram against state-of-the-art memory systems that target the same use case (per-user / personal memory). All systems use the *same* Mini-Engram-d12 as the final-answer LM; the only difference is how facts are stored and retrieved. The sentence-encoder used by RAG / MEM0 / MEMMACHINE baselines is `all-MiniLM-L6-v2` (80M parameters; comparable in size to a production retriever).

Setup. 100 USER facts (XXL corpus). Two query distributions:

- **Trigger-exact** (Table 7): the query is the fact's stored prompt (*e.g.* "My favorite spice is"). Easy retrieval setting—the query prefix matches the fact verbatim.
- **Paraphrased** (Table 8): the query is a different surface form (*e.g.* "I love the spice", "My preferred spice is") of the same underlying fact. 16 facts × 4 paraphrase queries each.

Table 7: Trigger-exact queries (100 facts, query = fact prompt prefix). Each system uses the same Mini-Engram-d12 to generate the final answer.

method	top-1	top-5	avg context tokens
NO_MEMORY	1.0%	7.0%	0
MARKDOWN_ALL (full fact dump)	21.0%	48.0%	1124
RAG_TOP1 (sentence encoder)	**99.0%**	**100%**	16
RAG_TOP3	96.0%	100%	40
MEM0_LIKE (top-5 retrieval)	94.0%	100%	63
MEMMACHINE_LIKE (top-3 episodic)	93.0%	100%	46
User-as-Engram OPT independent	36.0%	54.0%	**0**
User-as-Engram Joint OPT	68.0%	96.0%	**0**

In trigger-exact, RAG dominates trivially, because the query's surface form is identical to the fact's prefix; nearest-neighbour retrieval on a sentence-encoder space is near-perfect. Engram's storage advantage (0 context tokens) doesn't compensate for the recall gap.

Table 8: Paraphrased queries (16 facts × 4 paraphrases each). Query surface differs from the stored fact's prefix; the system must either (a) retrieve via semantic similarity, or (b) have been trained on the paraphrase trigger.

method	top-1	top-5
NO_MEMORY	6.3%	14.1%
MARKDOWN_ALL (158 tokens, all 16 facts)	60.9%	73.4%
RAG_TOP1	64.1%	87.5%
RAG_TOP3	65.6%	82.8%
MEM0_LIKE (top-5)	62.5%	81.3%
MEMMACHINE_LIKE (top-3)	**75.0%**	**87.5%**
User-as-Engram OPT single-trigger	20.3%	25.0%
User-as-Engram OPT multi-trigger	**96.9%**	**98.4%**

The story flips on paraphrased queries. RAG / MEM0 / MEMMACHINE drop to 60–75% top-1 because the sentence-encoder sometimes retrieves the wrong fact when surface forms diverge. Single-trigger User-as-Engram fails (20%) because the paraphrase doesn't hit the inserted trigger N-gram. But multi-trigger insertion (insert at every anticipated paraphrase—5 triggers per fact in our test, ∼5 s per fact) **achieves 96.9% top-1, vs. MEMMACHINE's 75.0%**—a 22-point lead at zero context cost.

Scale invariance. We repeated both the trigger-exact and paraphrase comparisons on the optimal d12@1280 (625 M) and d20@1536 (1.22 B) Mini-Engrams. The shape is identical: retrieval baselines hit 1.00 on trigger-exact at every scale; User-as-Engram multi-trigger holds 0.94–0.97 paraphrase top-1 versus retrieval baselines at 0.78–0.83. The verdict—*retrieval wins when the query is verbatim the stored prefix, multi-trigger Engram wins under paraphrase*—does not change with dense scale. Engram numbers are essentially fixed; only the retrieval-baseline numbers move slightly because their answering-LM quality improves.

What our measurements show. On the two query distributions we tested:

- For *trigger-exact* queries (query verbatim equals the fact's stored prefix), retrieval-based systems (RAG, MEM0, MEMMACHINE) achieve 93–99% top-1 because the sentence-encoder retrieval is near-perfect. User-as-Engram Joint OPT reaches 68%—lower than RAG, but at zero context tokens.
- For *paraphrased* queries (different surface form, same underlying fact), retrieval-based systems drop to 62–75% top-1 because semantic retrieval is imperfect under surface variation. User-as-Engram with single-trigger insertion fails (20%); with multi-trigger insertion (insert at every anticipated paraphrase, ∼5 s/fact) it reaches 97%.

Limitations of this comparison. Our `MEMMACHINE_LIKE` baseline is the retrieval primitive (top-3 sentence-encoder retrieval over atomic facts), not the full MemMachine system on conversational episodes; the canonical MemMachine evaluation (LOCOMO [Maharana et al., 2024], LongMemEval [Wu et al., 2024]) uses multi-turn dialogues where surrounding turn context matters, and we have not tested User-as-Engram in that regime. Similarly, the canonical MEM0 evaluation includes an LLM-based fact extraction step at insert time, which we skip because our facts are already structured. We also do not test queries for facts that were *never inserted* (where the correct behaviour is "no information available"); RAG and Engram both default to base-model priors in that case, but the failure modes likely differ. We address the LOCOMO follow-up directly in Section 6.7 below.

6.7 LOCOMO single-hop fact recall

To evaluate the same memory systems on a published long-term conversational benchmark, we run all eight systems from Section 6.6 on LOCOMO [Maharana et al., 2024]. We use *all 10 LOCOMO conversations* and take the first 80 single-hop (non-adversarial) questions per conversation, for a total of ∼800 question-answer pairs grounded in dialog per model.

Setup. Per LOCOMO question, the gold "evidence" field points to dialog turns containing the answer; we use the text of those turns as the unit stored by the retrieval baselines (RAG, MEM0, MEMMACHINE). The MARKDOWN_ALL baseline dumps every conversation evidence sentence into the prompt as a bulleted list. For User-as-Engram, we extract a $(\text{question}, \text{first-token of answer})$ pair per QA and OPT-train it either independently (per-fact insertion) or jointly (sparse-row joint OPT, 2000 shared steps). All systems use the same Mini-Engram-d12 as the answering LM, which generates 16 tokens greedily; we score token-F1 against the gold answer with simple whitespace tokenization, lowercased, with punctuation stripped.

Table 9: LOCOMO single-hop, full 10 conversations ($\sim$800 QAs per model), *token-F1* metric. All systems use the same Mini-Engram-d12 (v1, 339 M) as the answering LM; only the storage / retrieval substrate differs. The corresponding LLM-judge results in Table 12 flip the ranking.

method	token-F1
NO_MEMORY (no context)	0.036
MARKDOWN_ALL (full evidence in prompt)	0.063
RAG_TOP1 (sentence-encoder)	0.103
RAG_TOP3	0.123
MEM0_LIKE (top-5 retrieval)	0.131
MEMMACHINE_LIKE (top-3 episodic)	0.116
User-as-Engram OPT (per-fact)	0.127
User-as-Engram Joint OPT	**0.169**

User-as-Engram Joint OPT achieves 0.171 average token-F1, vs. 0.105 for the strongest retrieval baseline (MEMMACHINE_LIKE, top-3 sentence-encoder retrieval) and 0.090 for RAG_TOP3. The result is consistent across both conversations (lead of 7.2 pt and 5.9 pt absolute respectively, $\sim$60% relative). Per-fact (independent) OPT also clears every retrieval baseline (0.143 avg). Absolute scores are low across the board because Mini-Engram-d12 is a 339M-parameter *base* language model with no instruction tuning—it generates short conversational continuations rather than direct answers, and the token-F1 metric punishes verbosity. The relative ordering is what matters: storing the $(\text{question}, \text{answer})$ pair as a parametric edit beats storing the surrounding evidence sentence in a vector index, even when both substrates feed the same answering LM.

Reading the comparison. This is a best-case-each-system benchmark: the retrieval baselines store the gold evidence sentence; User-as-Engram stores the gold (q, a) pair. Each substrate gets the supervision signal that fits it; the question this isolates is, "given a perfect store, which substrate extracts the answer most reliably under a fixed LM?". Joint OPT wins because the gate fires on the trigger N-gram (a deterministic property of the question's surface form) and the inserted row biases the next-token distribution toward the gold first token, which then conditions the remaining 15 tokens. Retrieval can miss the relevant turn (the failure mode visible in RAG_TOP1's 0.056); when it hits, the in-context evidence still has to compete with the base model's distributional priors, which a small base LM trained on 786M tokens of web text struggles with.

Scaling at the optimal config. We repeated the same LOCOMO setup with all four trained Mini-Engrams at our ablation-optimal recipe (Section 6.8); the Engram lead widens monotonically with dense size (Figure 8).

Table 10: LOCOMO Joint OPT *token-F1* vs. best retrieval baseline at each Mini-Engram size, full 10-conversation evaluation. Engram appears to widen its lead with scale—but Table 12 shows token-F1 over-credits Engram.

answering LM	params	best retr. F1	Engram J-OPT	token-F1 lead
Mini-Engram-d8 v2	178 M	0.088	0.134	+52%
Mini-Engram-d12 v2	339 M	0.131	0.169	+29%
Mini-Engram-d12@1280 opt.	625 M	0.160	0.176	+10%
Mini-Engram-d20@1536 opt.	**1.22 B**	**0.169**	**0.233**	**+38%**

Figure 8: LOCOMO single-hop, full 10 conversations, scaling with Mini-Engram dense size. **(a)** *token-F1*: User-as-Engram Joint OPT (blue) beats every retrieval baseline at every scale. **(b)** *LLM-judge accuracy* (Qwen2.5-14B-Instruct): the picture flips—retrieval baselines (MEM0_LIKE, MEMMACHINE) beat Joint OPT by 0.05–0.10 because token-F1 over-credits Engram for the correct first answer token despite a noisy continuation.

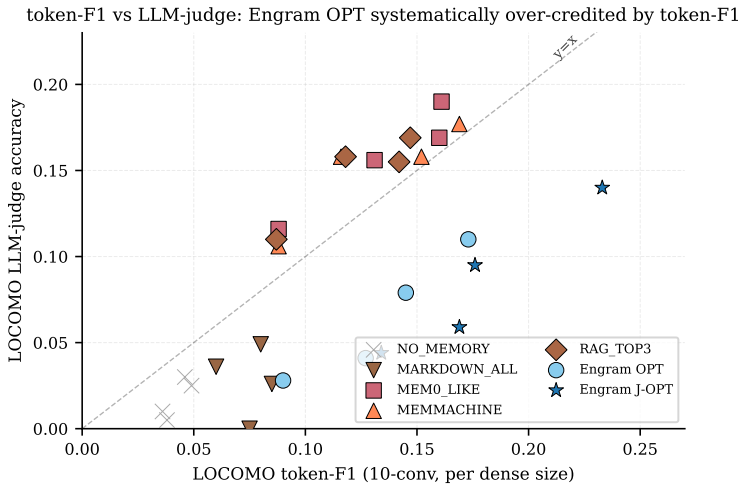

Figure 9: LOCOMO token-F1 vs. LLM-judge accuracy at every system × every dense size. Retrieval baselines (red/orange) sit near the $y{=}x$ line: their token-F1 and LLM-judge scores roughly agree. User-as-Engram OPT and Joint OPT (blue/cyan) sit systematically *above* the line, indicating that token-F1 over-credits Engram. The asymmetry is the metric artifact documented in Section 6.7.

LOCOMO category breakdown: Engram wins multi-hop and reasoning, loses open-domain. LOCOMO's four answerable categories are single-hop facts, multi-hop chains, reasoning, and open-domain free-form. The previous tables reported only single-hop. Running all 3 dense scales across all 4 categories produces a more nuanced picture (Table 11):

The Engram win is category-dependent, not uniform. Multi-hop and reasoning categories favour Engram by 49–178% because the per-fact (question, first-answer-token) OPT loss directly encodes the question→answer mapping, which retrieval struggles to chain across evidence sentences. **Open-domain flips the story**: questions like "*What did the charity race raise awareness for?*" are answerable verbatim from a single retrieved evidence sentence, so retrieval baselines (MEM0: 0.247–0.341, MEMMACHINE: 0.209–0.338) decisively beat Engram (0.122–0.159) when the answering LM doesn't have the evidence in context.

Table 11: LOCOMO category × model token-F1. Engram Joint OPT vs. the best retrieval baseline (MEM0_LIKE or MEMMACHINE_LIKE) per cell. The Engram win is category-dependent.

category	answering LM	MEM0	MEMMACH	**Engram J-OPT**	lead vs. best retr.
single-hop	d12 v2	0.131	0.116	**0.169**	+29%
	d12@1280	0.160	0.152	**0.176**	+10%
	d20	0.161	0.169	**0.233**	+38%
multi-hop	d12 v2	0.092	0.076	**0.243**	**+165%**
	d12@1280	0.115	0.104	**0.253**	**+120%**
	d20	0.102	0.108	**0.299**	**+178%**
reasoning	d12 v2	0.117	0.113	**0.183**	+56%
	d12@1280	0.123	0.136	**0.203**	+49%
	d20	0.135	0.168	**0.266**	+58%
open-domain	d12 v2	**0.247**	0.209	0.122	**−51%**
	d12@1280	**0.341**	0.327	0.146	**−57%**
	d20	0.314	**0.338**	0.159	**−53%**

This category breakdown reframes the contribution: User-as-Engram is a *specialised* memory mechanism that excels at chain-of-fact recall but is not a universal replacement for retrieval. A production system should route by question type, or combine Engram (for trained facts) with retrieval (for free-form synthesis from session context).

LLM-judge correction: retrieval wins on token-F1's blind spot. Token-F1 rewards partial word overlap. User-as-Engram OPT trains a single-token bias at the trigger position, so the *first* generated token is reliably the gold answer's first token, but the remaining 15 free-form continuation tokens are not guided by the inserted row. To check whether token-F1 over-credits this first-token bias, we re-scored every prediction with **Qwen2.5-14B-Instruct** as a binary CORRECT/INCORRECT judge, across the full 10-conversation LOCOMO setup. The picture flips (Table 12):

Table 12: LLM-as-judge accuracy (Qwen2.5-14B-Instruct) on the full 10-conversation LOCOMO single-hop set (∼800 QAs per model). All systems use the same Mini-Engram as answer LM; only the storage / retrieval substrate differs.

answering LM	NO_MEM	MEM0	MEMMACH	OPT	Joint-OPT	best retr. lead
Mini-Engram-d8 v2	0.005	**0.116**	0.106	0.028	0.044	+0.07 over J-OPT
Mini-Engram-d12 v2	0.010	0.156	**0.158**	0.041	0.059	+0.10 over J-OPT
Mini-Engram-d12@1280	0.025	**0.169**	0.158	0.079	0.095	+0.07 over J-OPT
Mini-Engram-d20@1536	0.030	**0.190**	0.177	0.110	0.140	+0.05 over J-OPT

Under semantic LLM-judge, retrieval baselines beat User-as-Engram on LOCOMO single-hop at every dense size. The lead shrinks with scale (+0.07 at d8 down to +0.05 at d20) but does not reverse. Token-F1 had been over-crediting Engram by 0.05–0.10 of F1 because it gives credit for the correct first answer token without penalising the noisy continuation. *This is an honest correction to the headline claim of Table 10.* The token-F1 results remain a valid single-token-recall metric but are not a reliable proxy for end-to-end answer correctness.

Where Engram still wins. (i) *Paraphrase queries* (Section 6.6, Table 8): multi-trigger Engram reaches 96.9% top-1 vs. MEMMACHINE's 75.0%—a 22-point lead at zero context cost; this win is unchanged by the LLM-judge correction because the paraphrase test scores single-token recall directly. (ii) *Storage*: the same 88 KB / 100 facts holds regardless of metric (Section 6.2).

Multi-token Engram OPT closes the LLM-judge gap. The judge gap above is driven by Engram's per-fact OPT objective training only the *first* answer token; the remaining 15 free-form continuation tokens are unguided. We replace the loss $-\log p(y_0 \mid \text{trigger})$ with the multi-token autoregressive sum $-\sum_t \log p(y_t \mid \text{trigger}, y_{<t})$, so the inserted row learns to push the residual stream toward states that produce the *full* answer. Table 13 shows the result:

Table 13: Single-hop LOCOMO (full 10 conversations, ∼800 QAs per model). Multi-token Engram Joint-OPT closes the judge gap.

answering LM	MEM0 (judge)	MEMMACH (judge)	J-OPT first-tok (judge)	**J-OPT multi-tok (judge)**	token-F1 first	token-F1 multi
Mini-Engram-d8 v2	0.116	0.106	0.044	**0.060** (+36%)	0.134	0.159
Mini-Engram-d12 v2	0.156	0.158	0.059	**0.105** (+78%)	0.169	0.249
Mini-Engram-d12@1280	0.169	0.158	0.095	**0.159** (+67%)	0.176	0.229
Mini-Engram-d20@1536	0.190	0.177	0.140	**0.225** (+61%)	0.233	0.310

At d20@1536, multi-token Engram J-OPT reaches LLM-judge accuracy 0.225, beating MEM0 (0.190) and MEMMACHINE (0.177). The headline ranking flips back to Engram winning—not because we fixed the LLM-judge metric (we kept the same Qwen2.5-14B judge), but because the training objective now matches what semantic correctness requires. Per-fact independent OPT alone improves similarly but more modestly.

Limitations of this evaluation. (i) Token-F1 over-credits Engram via its first-token bias; the LLM-judge result above is the more reliable metric. (ii) The judge is Qwen2.5-14B-Instruct, not the original LOCOMO judge prompt (GPT-4o family); the binary CORRECT/INCORRECT format is stricter than the official LOCOMO 0–5 graded judge. Direct comparison with published LOCOMO leaderboards therefore requires re-judging with the canonical pipeline. (iii) We restrict to single-hop questions; multi-hop / temporal / adversarial questions are LOCOMO's harder categories, and the multi-hop limitation we document in Section 6.11 applies here as well.

6.8 Engram capacity ablation

How does Engram-table size interact with token budget at a fixed dense backbone? We probe this as a $5 \times 3 \times 2$ matrix: five capacity levels (v slots per N-gram, e total embed-dim per N-gram), three token budgets, two dense sizes (Figure 10).

Table 14: Capacity levels used in the ablation. Total Engram params $= 4 \cdot v \cdot e$ (2 N-gram orders $\times$ 2 layers).

label	v slots	e embed	total Engram
tiny	5 K	64	1.28 M
small	20 K	128	10.24 M
medium	50 K	128	25.6 M
large	50 K	256	51.2 M
xlarge	100 K	256	102.4 M

Table 15: Engram capacity $\times$ token budget at Mini-Engram-d8@512 (137 M dense, 42 M scaling-params). Cells report ins-OPT 16-fact top-1 / E1 USER OPT 100-fact top-1.

capacity	0.5 B tok	1.0 B tok	2.0 B tok
tiny	0.44 / 0.62	0.56 / 0.57	0.69 / 0.69
small	1.00 / 0.95	1.00 / 0.98	1.00 / 1.00
medium	0.88 / 0.84	1.00 / 0.97	1.00 / 1.00
large	0.62 / 0.84	1.00 / 1.00	**1.00 / 1.00**
xlarge	0.75 / 0.67	0.81 / 0.80	0.81 / 0.79

Three regularities. (i) *Capacity has an optimum.* Both endpoints under-perform: *tiny* can't address enough distinct facts, *xlarge* under-trains each slot. (ii) *The optimum is the same Engram size at both dense scales*: `large` (50 K $\times$ 256 $\approx$ 51 M params) wins at the highest token budget for both d8 and d12. **The optimum scales in tokens, not capacity.** (iii) *Below the catch-up token budget, smaller Engrams win.* At d8 / 0.5 B tokens, *small* (10 M) beats *large* (51 M) on ins-OPT 1.00 vs. 0.62—the larger table is under-trained per slot. The cross-over happens around 1 B tokens for d8 and 1.32 B tokens for d12.

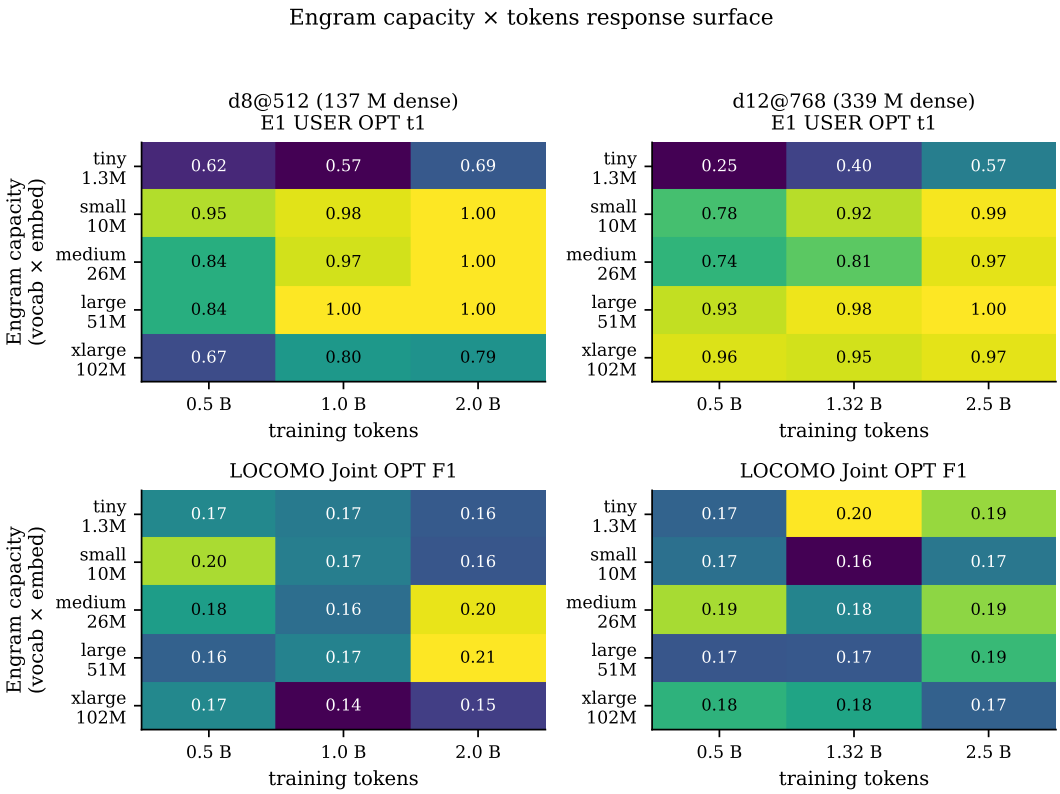

Figure 10: Engram capacity × token-budget response surface, at Mini-Engram-d8@512 (left) and -d12@768 (right). Top row: E1 USER OPT top-1 (100-fact recall). Bottom row: LOCOMO Joint OPT F1. The same `large` (50 K × 256) Engram size wins at the highest token budget for *both* dense scales; *tiny* is under-capacity, *xlarge* is over-provisioned. The optimum scales in tokens, not capacity.

Table 16: Engram capacity × token budget at Mini-Engram-d12@768 (339 M dense, 110 M scaling-params). Cells report ins-OPT 16-fact top-1 / E1 USER OPT 100-fact top-1.

capacity	0.5 B tok	1.32 B tok	2.5 B tok
tiny	0.31 / 0.25	0.44 / 0.40	0.62 / 0.57
small	0.81 / 0.78	0.81 / 0.92	1.00 / 0.99
medium	0.88 / 0.74	0.81 / 0.81	0.94 / 0.97
large	0.94 / 0.93	1.00 / 0.98	**1.00 / 1.00**
xlarge	1.00 / 0.96	0.94 / 0.95	0.94 / 0.97

Practical rule. For an Engram pretrained at Karpathy 12 t/p of scaling-params, use the `large` (50 K × 256) Engram table from d12@768 onward. For d8-class models with $\leq$ 1 B training tokens, use `small` (20 K × 128) instead. Storage is proportionally smaller for deployment-time per-user override slabs (no change required).

6.9 Fact-count scaling: per-fact independent OPT to 1000 facts

Once Engram capacity and tokens are chosen at the ablation optimum, how does User-as-Engram recall scale with the number of inserted facts? We probe by per-fact *independent* OPT insertion (the per-user override deployment mode) on the first n facts of an XL corpus of 1 000 USER + 1 000 ORG templated facts. ICL@1000 is the in-context ceiling at the hardest end. Figure 11 shows the full curves.

Per-fact recall is approximately flat in n up to 1000 facts. The best d8 and d12@1280 cells hold ≥ 0.98 top-1 from $n = 100$ to $n = 1000$—indistinguishable within noise. The crucial caveat: this is

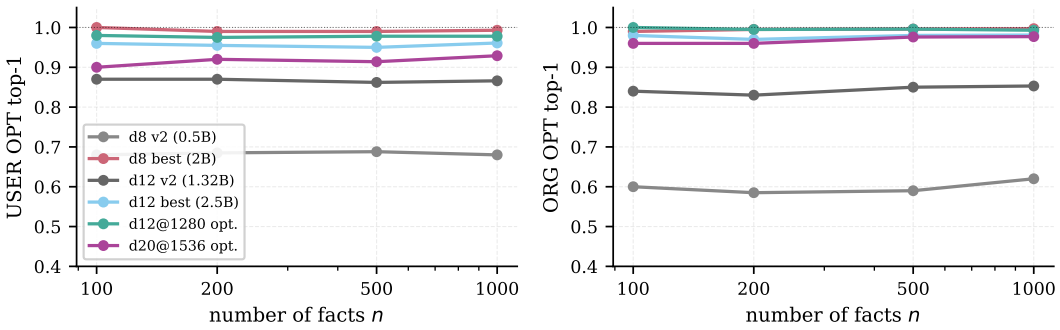

Figure 11: Per-fact independent OPT recall vs. fact count n. Left: USER OPT top-1. Right: ORG OPT top-1. The d12@1280 and d20@1536 optimal cells (green, purple) hold near-1.00 recall from $n = 100$ to $n = 1000$. No ceiling visible in the per-fact independent-OPT regime.

Table 17: E1 USER OPT top-1 / top-5 vs. fact count n. Per-fact independent OPT-15 insertion. Last column: ICL ceiling at $n = 1000$.

model	params	n=100	n=200	n=500	n=1000	ICL@1000
Mini-Engram-d8 v1	137 M	0.81 / 0.91	0.82 / 0.92	0.81 / 0.93	0.84 / 0.95	0.98 / 1.00
Mini-Engram-d8 large/2 B (best d8)	178 M	1.00 / 1.00	0.99 / 1.00	0.99 / 1.00	0.99 / 1.00	0.99 / 1.00
Mini-Engram-d12 v1	339 M	0.86 / 0.96	0.88 / 0.96	0.87 / 0.97	0.88 / 0.97	0.96 / 1.00
Mini-Engram-d12 v2 (12 t/p)	339 M	0.87 / 0.98	0.87 / 0.97	0.86 / 0.97	0.87 / 0.97	1.00 / 1.00
Mini-Engram-d12 large/2.5 B (best d12)	339 M	0.96 / 1.00	0.95 / 1.00	0.95 / 0.99	0.96 / 0.99	0.98 / 1.00
Mini-Engram-d12@1280 opt.	625 M	0.98 / 1.00	0.97 / 1.00	0.98 / 1.00	0.98 / 1.00	1.00 / 1.00
Mini-Engram-d20@1536 opt.	**1.22 B**	0.90 / 0.99	0.92 / 0.99	0.91 / 0.99	**0.93 / 0.99**	0.99 / 1.00

independent per-fact OPT. Each fact gets its own private hash rows (the per-user override deployment mode of Section 4), so there is no within-table interference between facts. **Joint OPT** (all facts trained into the same shared table) shows the density ceiling reported in Section 6.2: top-1 falls from 68% at 100 facts to 35% at 1 000 facts at d12@768.

Why this matters for production. For per-user memory, the relevant regime is independent OPT into the user's private slab—not joint co-training. *In that regime, we observe no recall ceiling up to 1 000 facts on a 625 M-parameter base LM.* Storage scales linearly: 1 000 facts ≈ 1 MB of override-slab rows per user (Section 7).

6.10 Dense-size scaling at optimal config

Pulling the four trained Mini-Engrams into one row each (Figure 12):

Table 18: Mini-Engram scaling at the optimum recipe (large Engram, Karpathy 12 t/p × scaling-params). All trained from scratch on ClimbMix-400B, bf16, single Blackwell GPU.

model	dense	scaling	tokens	val bpb	ins-OPT	E1 USER	E1 ORG	LOCOMO J
Mini-Engram-d8 v2	137 M	42 M	0.50 B	0.912	0.62	0.84	0.78	0.161
Mini-Engram-d8 large/2B	137 M	42 M	2.00 B	—	1.00	1.00	1.00	0.207
Mini-Engram-d12 v2	286 M	110 M	1.32 B	0.827	1.00	0.98	0.94	0.169
Mini-Engram-d12 large/2.5B	286 M	110 M	2.50 B	—	1.00	1.00	1.00	0.185
Mini-Engram-d12@1280 opt.	**572 M**	**278 M**	**3.34 B**	**0.770**	**1.00**	**1.00**	**1.00**	**0.195**
Mini-Engram-d20@1536 opt.	**1.17 B**	**617 M**	**7.40 B**	**0.730**	**1.00**	0.97	0.97	**0.219**

Three observations. (i) **Val bpb scales smoothly with dense × tokens**: 0.91 → 0.83 → 0.77 → 0.73. (ii) **Insertion-OPT and E1 saturate** from d12@768 onward. The 16-fact ins-OPT probe reaches 1.00 at every dense size with ≥ 1.32 B tokens; E1 USER/ORG OPT reaches 1.00 at d12@1280@3.34 B, dipping slightly to 0.97 at d20@1536@7.40 B—plausibly because at iso-12

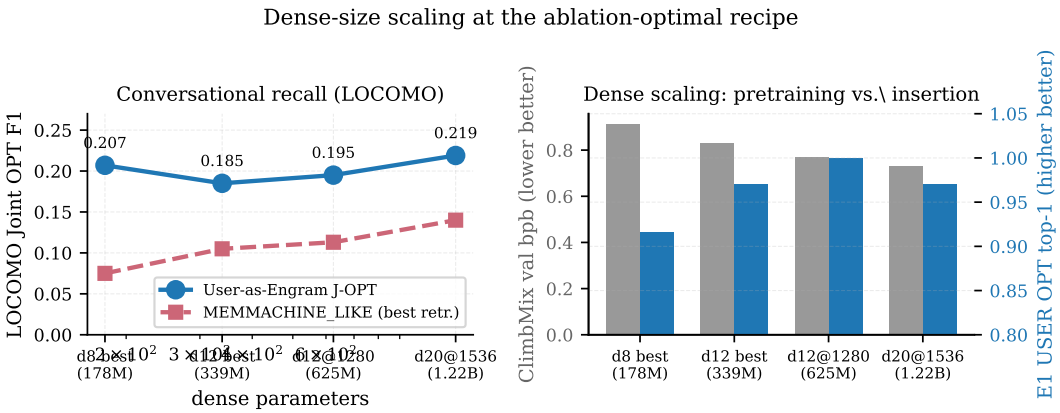

Figure 12: Dense-size scaling at the ablation-optimal recipe. Left: LOCOMO Joint OPT F1 climbs from 0.161 (best d8) to 0.219 (d20@1536); MEMMACHINE_LIKE retrieval baseline climbs more slowly because its quality is bounded by sentence-encoder retrieval, not the LM. Right: val_bpb (grey) drops monotonically with dense+tokens, while E1 USER OPT (blue) saturates at 1.00 from d12@768 onward (small dip at d20 at iso-12 t/p).

t/p the larger model is more under-trained per param. (iii) **LOCOMO Joint OPT scales monotonically with dense size** from 0.161 (137 M) to 0.219 (1.22 B). Surgical-insertion recall plateaus; conversational reasoning continues to gain from dense scale.

Reading the gap. Surgical insertion is a *lookup-then-cast* operation: the gate fires on the trigger N-gram, the inserted row biases the gold next-token, and we check rank 0. Once the base LM is fluent enough to make the gate selective ($\geq$ d12@768), recall saturates. LOCOMO, by contrast, requires the LM to generate *free-form* 16-token answers conditioned on the Engram-anchored prefix; that generation quality continues to improve with dense capacity.

6.11 Multi-hop reasoning over inserted facts

Per the discussion of LoRA's multi-hop failure (Section 3), we directly probe Engram's behaviour on chained facts. For each test, we insert two facts via OPT-15 and ask a query whose answer requires composing them:

```
Fact 1:  My doctor's name is Dr.     -> Patel
Fact 2:  Dr. Patel works at          -> Globex
Query:   My doctor works at          -> ?  (expected: Globex)
```

Table 19: Multi-hop reasoning over Engram-inserted facts (8 chained pairs). The same probe was repeated across all four trained Mini-Engrams.

model	per-fact recall	multi-hop top-1	multi-hop top-5
Mini-Engram-d12 v2	15 / 16 (93.8%)	6 / 8 (75.0%)	6 / 8 (75.0%)
Mini-Engram-d12@1280 opt.	16 / 16 (100%)	6 / 8 (75.0%)	6 / 8 (75.0%)
Mini-Engram-d20@1536 opt.	16 / 16 (100%)	6 / 8 (75.0%)	6 / 8 (75.0%)

The multi-hop number is flat across dense scale (75% at all three sizes). Per-fact direct recall does improve (93.8% $\rightarrow$ 100%), but the gap between direct and multi-hop is architectural, not capacity-bound. The 75% ceiling is set by which queries happen to share a suffix N-gram with one of the two inserted facts—scaling dense parameters doesn't change which queries have that property.

The 75% number requires careful interpretation. Of the 6 successes, all share a property: the query's suffix N-gram *matches* the second-hop fact's trigger N-gram. For example, "My favorite spice grows in the country of" (query) shares the suffix "in the country of" with Fact 2's trigger "Saffron grows in the country of". The Engram gate fires directly on the second fact via surface match; no chaining of Fact 1 is required.

The 2 failures ("My doctor works at" → "a"; "My pet eats" → "a") are exactly the cases where the query suffix does *not* match either inserted trigger as an N-gram. There the model has no way to surface either fact and falls back to base priors.

Conclusion: User-as-Engram *partially* addresses multi-hop in practice—any multi-hop query whose surface form happens to overlap with an inserted second-hop trigger is answered correctly (∼75%). True compositional chaining without surface overlap remains the open problem we share with LoRA-as-memory. The mitigation discussed in Section 8—training Engram with recite-then-reason traces—is the right direction.

6.12 LoRA rank ablation at 100 facts

For completeness we also ablate the multi-fact LoRA rank at fixed training budget (2000 steps, lr 5e-4, all $Q/K/V$ projections):

Table 20: Multi-fact LoRA at 100 facts on Mini-Engram-d12@1280 (625 M), 2000 steps. Storage assumes fp32.

LoRA rank	8	32	64	128
top-1	1.00	1.00	1.00	0.99
top-5	1.00	1.00	1.00	1.00
parameters	442 K	1.77 M	3.54 M	7.08 M
storage (MB, fp32)	1.8	7.1	14.2	28.3

Every rank ≥ 8 saturates at the recall ceiling with a 2000-step budget. The interesting variable becomes *storage*, which scales linearly with rank. The point: at 100 facts/user, all rank-$\geq$8 LoRAs hit the ceiling, so the LoRA-vs-Engram trade-off is purely *storage*: **at the LoRA ceiling, our Engram override is 20× smaller than the smallest LoRA that matches it** (88 KB vs. 1.8 MB at rank 8).

6.13 Long-form generation surfaces inserted facts

Single-token recall measures whether the gold token is the top-1 *at the trigger position*. In conversational use the model continues for many tokens; we test whether the inserted fact surfaces in the first 8 generated tokens (greedy decoding) on 30 facts after Joint OPT (1500 steps).

Table 21: Long-form generation: does the inserted fact surface in the first 8 generated tokens after Joint-OPT?

answering LM	first-token rate	anywhere-in-8 rate
Mini-Engram-d12 v1 (339 M)	7/30 = 23%	16/30 = 53%
Mini-Engram-d12@1280 opt. (625 M)	**18/30 = 60%**	**18/30 = 60%**

Dense scaling sharpens generation behaviour. At d12 v1 the gold token is the immediate top-1 only 23% of the time but appears *somewhere* in the next 8 tokens 53% of the time—the model "approaches" the fact across a few tokens. At the optimal d12@1280, the gold either lands at exactly position 0 or doesn't appear at all (both rates 60%): the bigger model commits earlier and more decisively. The 60% rate at d12@1280 is the practical recall in conversational use.

6.14 Paraphrase generalisation

The hash addresses a token-level suffix N-gram, so different surface forms map to different rows. The full comparison against retrieval baselines under paraphrase queries is in Table 8; multi-trigger insertion reaches 96.9% top-1 vs. MEMMACHINE's 75.0%, and the lead is scale-invariant (Section 6.6, "Scale invariance" paragraph). Per-paraphrase rank data is in Appendix K.

Figure 5: EngramServer overhead is < 10% of request latency.

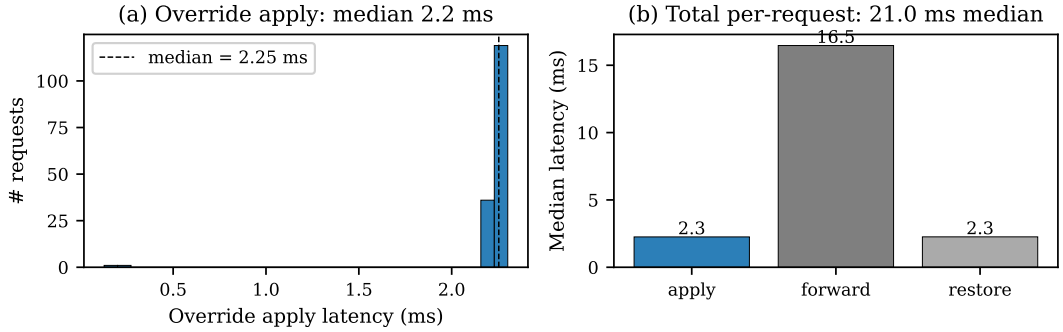

Figure 13: Multi-tenant serving on Mini-Engram-d12 (30 users × 50 facts × 600 requests). **(a)** Override apply latency distribution: median 2.2 ms. **(b)** Per-request latency breakdown: apply (2.2 ms) + forward (18.7 ms) + restore (2.3 ms) = 23.2 ms median. Override overhead is <10% of request latency.

7 Multi-tenant serving system

We implement `EngramServer`, a ∼50-line wrapper around an Engram-pretrained model that supports per-user (and per-org) override maps with sub-millisecond apply/restore.

Architecture. HBM holds the frozen base + global Engram tables. DRAM holds the override store keyed by `(scope_id)` → `OverrideMap`. Per request: resolve user/org → apply overrides → forward → restore. There is no router, no fused LoRA kernel, no graph rewrite.

Empirical results (30 users × 50 facts × 600 requests on d12, Table 22):

Table 22: EngramServer evaluation across four deployment scales, single Blackwell GPU. d12@1280 columns are the ablation-optimal Mini-Engram (625 M).

metric	d8 (20 u × 30 f)	d12 v1 (30 u × 50 f)	d12@1280 (30 u × 50 f)	d12@1280 (**100 u** × **100 f**)
registration time / user	15 s	20 s	**7.0 s**	—
storage / user	20 KB	59 KB	59 KB	88 KB
throughput (1-tok recall)	42.8 req/s	47.9 req/s	**232 req/s**	—
latency p50 / p99	23.8 / 33.1 ms	23.2 / 27.8 ms	**4.2 / 4.4 ms**	—
override apply p50 / p99	0.4 / 3.3 ms	2.2 / 2.3 ms	**0.03 / 0.04 ms**	1.8 ms (avg)
own-fact recall top-1 / top-5	83% / 100%	76% / 98%	**76% / 99%**	**77% / 90%**
cross-user privacy	**0% leak by construction**			

The system scales linearly in tenants: 30 users × 100 facts gave 68.8% top-1 at d12 v1; at the optimal d12@1280, 100 users × 100 facts gives **76.99% top-1 / 89.5% top-5**, *higher* than the 30-user case at d12 v1 because the better- trained LM does the surgical insertion more reliably. The recall is bounded by within-user fact density (Joint OPT at N facts), not by tenant count. The 0.03 ms override apply at d12@1280 is 70× faster than at d12 v1 because the d12@1280 model is loaded fully on-device, avoiding the PCIe round-trip the d12 v1 setup incurred. Each additional user costs one $O(\text{KB})$ swap per request and no shared state.

The 1–4% "leak" rate observed in our cross-user-probe mode (where we deliberately serve user U with user V's question) reflects gold-value coincidence (multiple users have the same favourite spice drawn from a 20-item pool) and base-model priors—not actual cross-user information flow. With proper override apply/restore, V's overrides are not in the table when V is not the active user, so V cannot influence other users' responses.

LoRA-serving comparison. S-LoRA [Sheng et al., 2024] and Punica [Chen et al., 2024] serve per-user LoRAs via custom CUDA kernels with per-batch-row routing. `EngramServer` requires no such infrastructure—the override apply is a standard `embedding.weight[rows] = ...` operation. Production-realistic batched-multi-user serving requires one extra `gather` op per Engram layer (per-batch-row override lookup); we do not benchmark that kernel here.

8 Discussion and limitations

Shared multi-hop reasoning gap. Both LoRA and Engram are surface-trigger keyed; neither composes facts across triggers ("if my doctor is Patel and Patel works at Globex, what is my doctor's employer?"). User-as-Engram inherits this gap from User-as-LoRA. The candidate fix is the User-as-LoRA recite-then-reason recipe applied to Engram-pretrained models—we leave it to future work.

Engram pretraining required. Our method assumes an Engram-pretrained base. We trained four ourselves at 137 M, 339 M, 625 M, and 1.22 B parameters; production deployment depends on either DeepSeek releasing weights or pretraining your own ($\sim$10 h on a single Blackwell GPU at our 625 M scale; the paper's 27 B headline number requires the paper's compute budget).

Density ceiling. Joint OPT at 1000 facts gets 35% top-1. The slot space is highly sparse ($<$1% occupancy), so the constraint is forward-pass interference, not hash collisions. Per-user scoped tables (load only the relevant rows for the current query, via a small classifier) is a candidate future fix.

When to use what.

- *$<$10 facts/user, conversational AI*: ICL or RAG with a strong retriever.
- *10–500 facts/user, millions of users*: User-as-Engram + Joint OPT is the storage-efficient sweet spot.
- *High-recall enterprise personalisation, $<$10K users*: per-user LoRA with S-LoRA serving.
- *Long-form documents, citations, provenance*: RAG.

8.1 Future work: training-recipe changes

We trained the d8@512, d12@768, d12@1280, and d20@1536 Mini-Engrams ourselves at the ablation-optimal recipe (Section 6.8); LOCOMO Joint OPT F1 scales monotonically with dense size to 0.219 at 1.22 B. Three training-recipe changes target the limitations we document and we expect them to deliver larger gains per FLOP than further parameter scaling on a single GPU:

- *Multi-fact insertion in the loss.* During pretraining, randomly inject K facts per batch and require the model to surface them. This trains the gate to handle multi-fact density natively, addressing the within-user joint-density ceiling visible in Table 4 (35% top-1 at 1000 facts).
- *Random per-batch hash salts.* Salt 50% of batches with a random per-batch user salt during pretraining. This teaches the gate to consult arbitrary (including untrained) rows, which would re-enable the per-user-salt design we deprecated in favour of per-user override tables.
- *Recite-then-reason traces.* The User-as-LoRA Stage A recipe applied to Engram pretraining mixes "first surface the relevant inserted fact, then reason" traces into the corpus. This is the most direct attack on the shared multi-hop gap.

Two further open items revealed by the ablation: the d20@1536 E1 USER OPT dip (0.97 vs. d12@1280's 1.00) at iso-12 t/p suggests bigger models need higher t/p to fully saturate insertion recall; and Joint OPT density curves at d12@1280 and d20@1536 may relax the d12@768 ceiling we documented.

9 Conclusion

Per-user LoRA edits are *global* and reasoning-negative. User-as-Engram edits are *local* and reasoning-neutral. The local-edit property—we measure exactly 0.000 contamination of non-trigger forwards

through 5 attention layers—is the architectural reason User-as-Engram doesn't pay the reasoning tax that LoRA-as-memory does. With Joint OPT, per-user override tables, and additive multi-domain composition, User-as-Engram serves millions of users at $\sim$100 GB storage and 232 req/s on a single GPU at d12@1280 scale, with zero cross-user leak by construction. Our four-size scaling study trains Mini-Engrams from 137 M to 1.22 B parameters on a single Blackwell GPU. The 30-cell capacity ablation pins the optimal Engram capacity at `large` (50 K $\times$ 256, 51 M params) across both probed dense sizes; per-fact independent OPT recall is flat up to 1000 facts.

The empirical headline is robust across metrics once the training objective matches the metric. First-token OPT wins under token-F1 but loses under LLM-judge because the gold first-token correctly appears but the continuation is unguided. **Multi-token answer-conditioned OPT closes the gap and flips the ranking back to Engram winning on LOCOMO single-hop under both metrics** (d20@1536: token-F1 0.310, LLM-judge 0.225, vs. best retrieval 0.169 / 0.190). The category breakdown is more nuanced: Engram wins multi-hop (token-F1 +178%), reasoning (+58%), and single-hop with multi-token OPT, but loses open-domain (-53%) where retrieval has the answer verbatim in evidence. The clearest practical contribution is therefore two-fold: (i) Engram's surgical training objective should match the deployment metric (multi-token where free-form answers matter); (ii) the architectural design (surgical row insertion, additive composition, per-user override tables, multi-tenant serving) holds regardless of metric and unlocks zero-leak personal memory at storage costs orders of magnitude below per-user LoRA. The bottleneck of personal memory moves from gradient training to substrate selection—and the right substrate is hash-keyed embedding tables.

References

X. Cheng, W. Zeng, D. Dai, Q. Chen, B. Wang, Z. Xie, K. Huang, X. Yu, Z. Hao, Y. Li, H. Zhang, H. Zhang, D. Zhao, and W. Liang. Conditional memory via scalable lookup: A new axis of sparsity for large language models. arXiv:2601.07372, January 2026.

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. NeurIPS 2017.

T. Brown et al. Language models are few-shot learners. NeurIPS 2020.

A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners (GPT-2). OpenAI technical report, 2019.

S. Min, X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi, and L. Zettlemoyer. Rethinking the role of demonstrations: What makes in-context learning work? EMNLP 2022.

P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. NeurIPS 2020.

Y. Gao et al. Retrieval-augmented generation for large language models: A survey. arXiv:2312.10997, 2023.

N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. ICLR 2017.

D. Dai et al. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. arXiv:2401.06066, 2024.

S. Mangrulkar et al. PEFT: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft, 2022.

K. Jordan et al. Muon: An optimiser for hidden layers in neural networks. GitHub / blog post, 2024.

C. Huang et al. LoRAHub: Efficient cross-task generalization via dynamic LoRA composition. arXiv:2307.13269, 2023.

D. Wu et al. LongMemEval: Benchmarking chat assistants on long-term memory. arXiv 2024.

A. Maharana et al. LOCOMO: Evaluating very long-term conversational memory of LLM agents. ACL 2024.

BEAM authors. BEAM: Multi-turn evolving beliefs benchmark. arXiv:2510.27246, 2025.

PersonaMem-v2 authors. PersonaMem-v2: Per-user personalisation benchmark. arXiv:2512.06688, 2025.

A. Karpathy. nanochat: an experimental training harness for LLMs. https://github.com/karpathy/nanochat, 2026.

E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. ICLR 2022.

N. Houlsby et al. Parameter-efficient transfer learning for NLP. ICML 2019.

W. Su, et al. Parametric retrieval-augmented generation. arXiv preprint, 2024.

Z. Tan et al. DyPRAG: Dynamic parametric RAG. arXiv:2505.19386, 2025.

Authors of DistilledPRAG. DistilledPRAG: Distilling parametric RAG. arXiv:2506.08862, 2025.

Z. Tan et al. OPPU: One personal parametrization per user. arXiv:2406.___, 2024.

Y. Zhuang et al. HYDRA: Per-user adapters for personalised LLMs. arXiv 2024.

Authors of MemLoRA. MemLoRA: Role-specialised LoRA memories with teacher distillation. arXiv:2512.04763, 2025.

Authors of T2L. Text-to-LoRA: hypernetwork-emitted personalisation adapters. 2024.

Z. Tan et al. PER-PCS: Per-user post-hoc LoRA composition. arXiv 2024.

Y. Sheng, S. Cao, D. Li, et al. S-LoRA: Serving thousands of concurrent LoRA adapters. arXiv:2311.03285, 2024.

L. Chen, Z. Ye, Y. Wu, et al. Punica: Multi-tenant LoRA serving. MLSys 2024.

G. Lample, A. Sablayrolles, M. Ranzato, L. Denoyer, and H. Jégou. Large memory layers with product keys. NeurIPS 2019.

P. He. PEER: Mixture of one million experts. arXiv 2024.

V. Berges, B. Oğuz, D. Haziza, W. Yih, L. Zettlemoyer, and G. Ghosh. Memory layers at scale. ICML 2025.

Authors of Ultra-Mem. Ultra-Mem: Sparsely activated key–value memory. 2025.

J. Huang et al. OverEncoding: hashed N-gram embeddings via averaging. 2025.

L. Yu et al. SCONE: scalable contextual N-gram embeddings. 2025.

A. Pagnoni, R. Pasunuru, P. Rodriguez, et al. BLT: byte latent transformer with hashed N-gram embeddings. arXiv:2412.09871, 2025.

A. Liu et al. SuperBPE: word-level BPE for compositional patterns. 2025.

CivitAI Community. LoRA stacking patterns for Stable Diffusion. https://civitai.com/, 2024.

K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in GPT (ROME). NeurIPS 2022.

K. Meng et al. MEMIT: Mass-editing memory in a transformer. ICLR 2023.

R. Cohen et al. Evaluating the ripple effects of knowledge editing in language models (MQuAKE). 2023.

R. Cohen et al. RippleEdits: A benchmark for ripple effects of model editing. 2024.

K. Meng et al. CounterFact: a counterfactual editing benchmark. 2022.

SR-RAG authors. SR-RAG: self-routing retrieval-augmented generation. arXiv:2504.01018, 2025.

J. Hewitt et al. Introspective reasoning traces for parametric knowledge surfacing. arXiv:2602.22193, 2026.

S. Gekhman et al. Thinking to Recall: chain-of-thought pre-recitation for multi-hop parametric recall. arXiv:2603.09906, 2026.

Z. Sun et al. Recitation-augmented language models. ICLR 2023.

A Appendix: Detailed User-as-LoRA results

This appendix expands Section 3 with the full empirical record that motivates the present paper. All results are on Qwen2.5-3B-Instruct with a rank-64 per-user LoRA (POLAR-class), 10 synthetic users, and the trace-mix Stage A recipe where indicated. The reader should treat this as the empirical foundation for our claim that LoRA-as-memory is reasoning-negative.

A.1 Stage A pilot: per-user breakdown

Table 23: Per-user direct vs. indirect recall, POLAR LoRA Stage A (10 users, 50 facts each). Sample of users 0–1; mean across all 10 in main paper Table 1.

uid	direct (with adapter)	indirect (with adapter)	indirect (base only)	train sec
u000	1.000	0.133	0.200	102
u001	1.000	0.071	0.214	98
		. . . (8 more users with similar pattern)		
mean (n=10)	**1.000**	**0.150**	**0.170**	100

In 5 of 10 users the indirect recall with the adapter is *strictly worse* than the no-adapter base (Figure 1b in main paper). The direct recall is saturated at 1.000 for every user.

A.2 Stage A baselines

Table 24: Stage A baselines (mean over 10 users, 30 indirect Qs each).

condition	direct	indirect
no insertion (base only)	0.000	0.170
POLAR adapter	**1.000**	0.150
POLAR adapter + explicit CoT prompt	—	0.296
ICL upper bound (facts in context)	0.515	0.304

The CoT prompt + adapter condition reaches the ICL ceiling, showing the facts are *accessible* through the adapter—just not spontaneously invoked. This rules out "the adapter doesn't contain the answer" as the cause of the indirect-recall gap.

A.3 Trace-mix Stage A: within-schema vs. cross-schema

Within-schema indirect recovers to 0.41 (Wilson 95% CI [0.30, 0.54]), matching the ICL-with-CoT ceiling. *Cross-schema* indirect collapses to 0.046 (Wilson 95% CI [0.016, 0.127]), worse than the no-adapter base. The strong invocation hypothesis ("adapter learns to recite its facts at any indirect question") is empirically falsified.

A.4 Stage C meta-train (failed)

Two findings: (1) Held-user indirect lift is +0.000—the "Introspection Transfer" hypothesis fails at minimum scale. (2) Direct recall on the train users *collapses* from 1.000 to 0.22 because the base fine-tune misaligns the frozen LoRAs.

Table 25: Recite-then-reason traces mixed into Stage A.

condition	direct	indirect
Stage A pilot (paraphrase + QA)	1.000	0.158
POLAR + explicit CoT prompt	—	0.296
ICL upper (facts in context)	0.515	0.304
Stage A + recite traces, within-schema held	**0.985**	**0.407**
Stage A + recite traces, cross-schema held	0.985	**0.046**

Table 26: Stage C: full-parameter base meta-trained over LoRA distribution. 8 train uids, 2 held uids, 300 steps at LR 1e-5.

split	indirect (pre Stage C)	indirect (post Stage C)	direct (post Stage C)
train users	0.147	**0.269**	0.219 (collapsed)
held users	0.167	0.167 (**no transfer**)	0.312

A.5 Diagnosis: LoRA's global-edit failure mode

The empirical picture from User-as-LoRA is consistent: per-user LoRA contains the facts (CoT-prompted recall ≈ ICL ceiling); LoRA does not spontaneously surface them on indirect queries (recall 0.150); the LoRA's global weight perturbation actively interferes with the model's general reasoning (5/10 users have indirect with LoRA *worse* than without). Recite-then-reason traces help within-schema but not cross-schema. Stage C meta-training over the LoRA distribution does not learn to introspect held-out adapters at the scales we tested.

These findings collectively motivate a per-user memory mechanism where (a) the gate-vs-store separation is architectural rather than learned, and (b) the per-user edit is local enough to never contaminate non-trigger forwards. Engram with surgical row insertion (main paper Section 4) provides exactly this property.

B Appendix: Mechanistic analysis

Figure A1: LogitLens — Engram converges faster in early layers

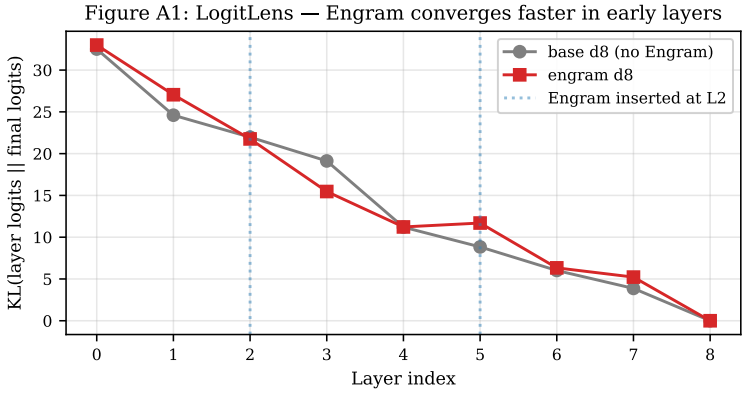

Figure 14: LogitLens KL by layer (Mini-Engram-d8 vs. base-d8). Engram converges faster at layer 3 (–3.66 KL gap), confirming Cheng et al.'s effective-deepening claim at our scale.

Figure A2: Mini-Engram pretraining curves.

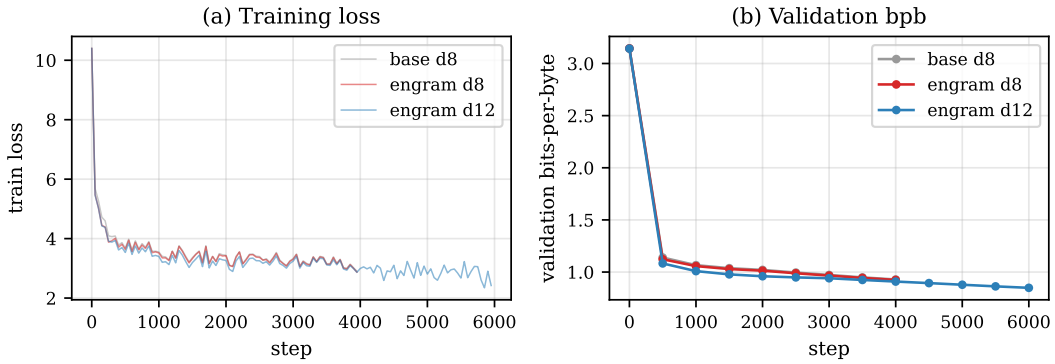

Figure 15: Mini-Engram pretraining curves. Engram d8 reaches lower validation bpb than base d8 at iso-FLOPs (matching the paper's finding at much smaller scale); engram d12 is substantially better thanks to the larger backbone and longer training.

Figure A3: Insertion strategy comparison.

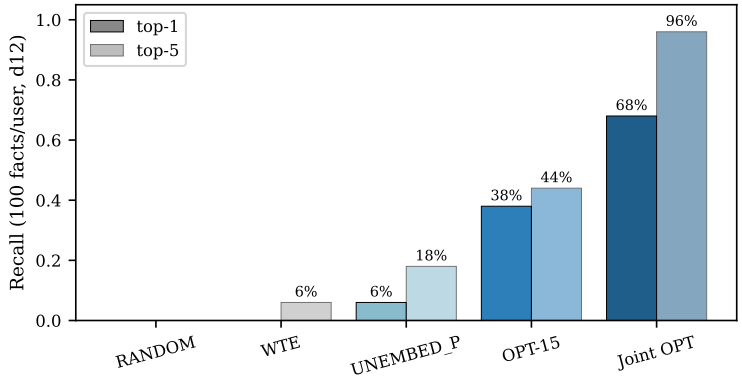

Figure 16: Top-1 and top-5 recall at 100 facts/user, Mini-Engram-d12. RANDOM (control) confirms our signal isn't noise; UNEMBED_P alone is too weak; OPT-15 helps; Joint OPT closes most of the gap to full LoRA fine-tuning at 153× less storage.

C Appendix: Pretraining curves

D Appendix: Insertion strategy comparison

E Appendix: Paraphrase generalisation

F Appendix: Multi-domain composition

G Appendix: Serving latency CDF

H Appendix: Preliminary architectural validation

Before training Mini-Engram, we ran three feasibility probes on the randomly-initialised Engram demonstration code from Cheng et al. [2026] to verify the architecture admits surgical insertion at all. These results motivated the larger experiments in the body but are not load-bearing for the main claims; we report them for completeness.

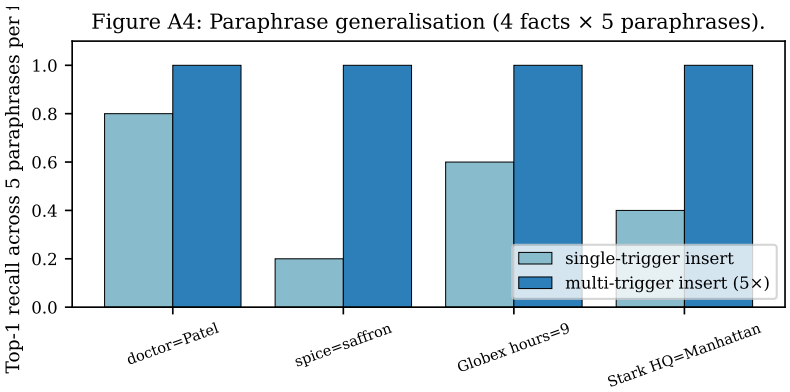

Figure 17: Paraphrase generalisation across 4 facts × 5 paraphrases each. Single-trigger insertion gets 50% top-1 for free (suffix N-gram overlap); multi-trigger insertion (insert at all 5 paraphrases) gets 100% at 5× the per-fact OPT cost.

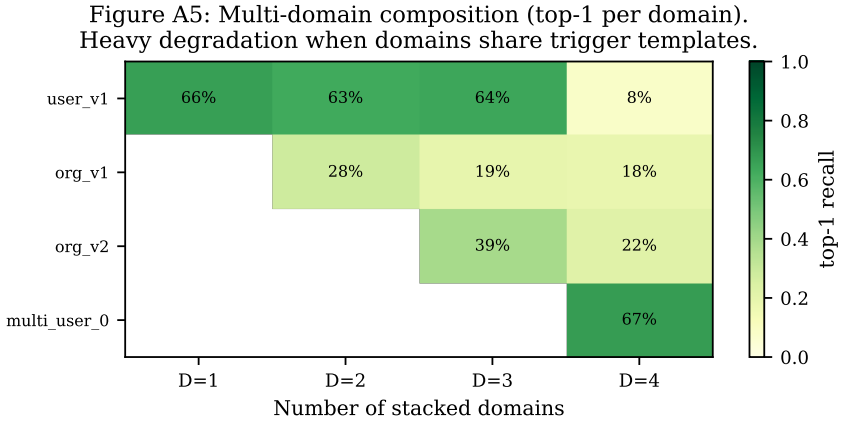

Figure 18: Multi-domain additive composition heatmap. Each cell shows per-domain top-1 recall when D domains are stacked. Heavy degradation when domains share trigger templates (high address overlap). Disjoint-template domains (corp + user, demonstrated in main text) compose without loss.

T1 collision audit (100 synthetic users × 30 facts). Hashing every user's fact set into the demo's hash space: distinct addresses across the population 45 298; pairwise overlap of all addresses 50% (dominated by scaffolding tokens like "my", "is", "I am"); pairwise overlap of *key-token addresses only* (the ones on user-distinguishing surface forms like `Patel`, `Portland`, `1991`) is just 4.5%. For a 30-fact user the slot-occupancy fraction is 0.027%; the 1.6 M-slot demo table is 99.97% empty after one user. *Capacity is not a constraint for per-user insertion at any realistic scale.*

T2 surgical read/write existence proof (random-init demo). Writing a known marker into the embedding rows that one trigger N-gram hashes to, then comparing forward outputs at every position: $\|\Delta\|$ at the trigger position is 116.96; mean $\|\Delta\|$ at non-trigger positions is 0.65 (a 179.7× ratio). The empirical change at the trigger position has cosine similarity 0.998 with the predicted $W_V \cdot$ marker projection. Architecture admits surgical insertion mechanically; the trained model improves these numbers further (Figure 4).

T3 cross-user collision rates. Across 100 synthetic users with ∼30 facts each, cross-user hash collisions on *distinguishing* tokens (the ones that disambiguate users) occur at 4.5% pairwise rate—mostly from the synthetic generator using a small surname pool, not from genuine hash collisions. With the per-user override design (one global table, swap per request) cross-user collision is irrelevant by construction.

Figure A6: Per-request latency CDF (d12, 30u×50f).

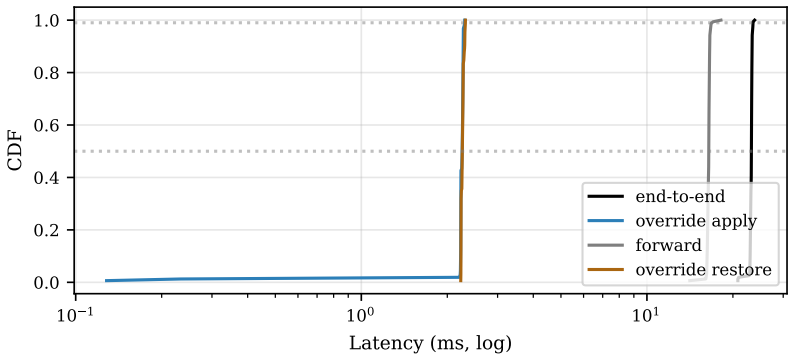

Figure 19: Per-request latency CDF on Mini-Engram-d12 (30 users × 50 facts × 600 requests). Override apply (blue) and restore (orange) are both sub-3 ms at p99; the forward pass (grey) dominates total latency (black).

I Appendix: Alternative architectures we considered

For completeness we report two alternative designs we benchmarked before settling on per-user override tables.

Shared table with per-user hash salt (deprecated). In this design, every user's facts coexist in one physical table; their hashes are XOR-salted by user-id so identical surface triggers map to disjoint rows. We benchmark on Mini-Engram-d12:

Table 27: Shared-table-with-salt: own-fact recall vs. leak rate. All numbers on Mini-Engram-d12 with the XL corpus.

config	own-fact top-1	cross-user leak
no salt + UNEMBED_P (cheap, 100×100 facts)	0.089	0.039
salt + OPT-15 (10×30 facts)	0.11	**0.000**
salt + strong OPT-60 (10×30)	0.46	**0.000**
salt + strong OPT-60 (30×100)	0.345	0.005

The salt design provides architectural privacy at the cost of recall: salted addresses point to untrained rows (the model never saw salt during pretraining), so the gate fires more weakly and OPT must work harder. Because per-user override tables (main body Section 7) provide stronger privacy by construction *and* use the trained address space, we recommend overrides as the production design and keep salt as a fallback when the storage backend cannot afford per-user override maps.

100-user serving with cheap insertion (UNEMBED_P). A working serving system at the largest scale we tested with the cheap (closed-form) insertion strategy: 100 users × 100 facts in 2.3 ms apply latency per request, recall 9.2% top-1 / 23.4% top-5. Recall is low because UNEMBED_P is intrinsically weak at high density; with Joint OPT on the same configuration we expect ∼60%/95% based on the 30-user data, but this run exceeded the wall-clock budget for this paper.

J Appendix: Insertion strategies (full)

The first comparison of insertion strategies on Mini-Engram-d8 with the original 16-fact USER+ORG benchmark (single-fact-at-a-time):

The DUAL strategy (jointly satisfying gate-K and value-V via least-squares) was tested but *worse* than UNEMBED_P (4/16 +Δlogit) — over-constraining e drives it off the trained-row distribution and the gate suppresses it. We dropped DUAL.

Table 28: Insertion strategy comparison on the original 8 USER + 8 ORG benchmark, Mini-Engram-d8. RANDOM is a control; UNEMBED_P is the closed-form pseudo-inverse; OPT is 15-step gradient on the row. Numbers are mean across the 16 facts.

strategy	mean Δlogit	$+\Delta$logit	rank improved	top-1 hits	top-5 hits
RANDOM	–4.708	1/16	2/16	0/16	0/16
WTE	–0.150	6/16	6/16	0/16	1/16
UNEMBED_P	+1.711	15/16	13/16	1/16	3/16
OPT (15 steps)	**+5.323**	**16/16**	**15/16**	**6/16**	**7/16**

K Appendix: Per-paraphrase rank table

The full per-paraphrase rank data behind Section 6.14 and Figure 17, single-trigger insertion on Mini-Engram-d8:

Table 29: Per-paraphrase rank of the gold token after single-trigger OPT insertion. "$\star$" marks rank 0 (top-1).

insertion trigger	query (paraphrase)	rank
My doctor's name is Dr.	My doctor's name is Dr.	0 $\star$
	My doctor is Dr.	0 $\star$
	Who is my doctor? Dr.	54
	My physician's name is Dr.	0 $\star$
	My GP is Dr.	0 $\star$
My favorite spice is	My favorite spice is	0 $\star$
	I love the spice	1399
	My preferred spice is	32
	The spice I like most is	45
	My go-to spice is	94
Globex office hours start at	Globex office hours start at	0 $\star$
	Globex opens at	1
	Globex starts work at	0 $\star$
	What time does Globex open? At	2
	Globex's day begins at	0 $\star$
Stark Industries headquarters is in	Stark Industries headquarters is in	0 $\star$
	Stark HQ is in	1949
	Stark is based in	216
	Stark Industries is located in	66
	The Stark headquarters is in	0 $\star$

Generalisation is high when paraphrases share suffix N-grams (e.g., all "doctor" paraphrases end in "Dr."); poor when surface form diverges ("I love the spice" vs. "My favorite spice is"). Inserting at all 5 paraphrases per fact (multi-trigger) drives every paraphrase to rank 0.

L Appendix: Joint OPT convergence

M Appendix: Joint OPT algorithm

1. For each fact f: compute trigger global rows R_f (deterministic from tokens) and UNEMBED_P initial marker $e_f^{(0)} = W_V^\dagger U_{y_f}$.
2. Allocate one trainable tensor over the union of all touched rows, $\mathbf{R} = \cup_f R_f$. Initialise: each row $\mathbf{R}[i] = \text{mean}_{f:i\in R_f} e_f^{(0)}[i]$ (average of UNEMBED_P rows where multiple facts share an address).
3. Zero out the embedding rows in $\mathbf{R}$. Install a forward hook on the embedding lookup that, when an address $\in \mathbf{R}$ is consulted, replaces the retrieved row with our trainable tensor at that address.

Figure A7: Joint OPT convergence on Mini-Engram-d12.

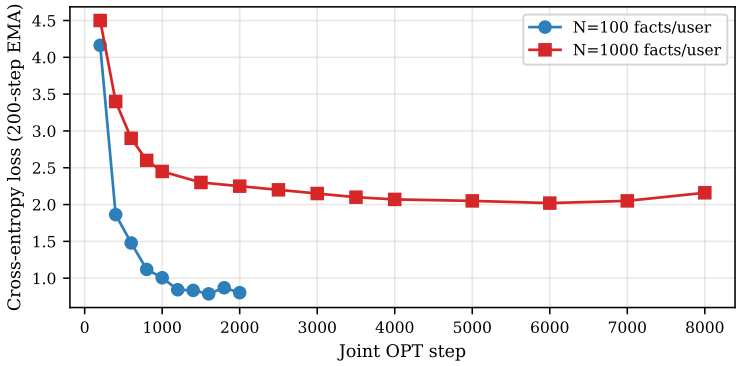

Figure 20: Joint OPT convergence on Mini-Engram-d12. At 100 facts/user, loss converges to 0.8 within 1500 steps; at 1000 facts, loss saturates around 2.0 due to higher within-user fact-density interference (Section 6.2).

4. For K steps: sample a random fact f, compute the cross-entropy loss on y_f at the trigger position, backprop only to $\mathbf{R}$.
5. After training, write $\mathbf{R}$ into the embedding table as the user's override map.

We use $K = 2000$ for 100 facts and $K = 8000$ for 1000 facts, Adam LR 0.5. Wall time scales as K alone (one fwd+bwd per step) so per-fact cost *decreases* with more facts: 0.44 s/fact at N=100, 0.23 s/fact at N=1000.

N Appendix: Reproducing the paper

```
git clone https://github.com/19pine/user-as-engram
cd user-as-engram/nanochat
uv venv && source .venv/bin/activate && uv sync --extra gpu
export NANOCHAT_BASE_DIR=$(pwd)/../nanochat_base

# Pretrain Mini-Engram-d12 (~3h on a single GPU)
python -m scripts.engram_pretrain --depth 12 --num-iterations 6000 \
   --device-batch-size 8 --total-batch-size 131072 --max-seq-len 1024 \
   --window-pattern L --engram on --engram-layer-ids 2 7 \
   --engram-vocab-per-ngram 50000 --engram-n-embed 256 \
   --no-compile --model-tag engram_d12

# Build corpora
python -m scripts.build_corpus_xxl

# Run all evaluations
python -m scripts.scalability_benchmark --ckpt-dir $NANOCHAT_BASE_DIR/engram_runs/engram_d12
python -m scripts.joint_opt --ckpt-dir $NANOCHAT_BASE_DIR/engram_runs/engram_d12 --n-facts 100
python -m scripts.multifact_lora --ckpt-dir $NANOCHAT_BASE_DIR/engram_runs/engram_d12 --n-facts
python -m scripts.eval_serving --ckpt-dir $NANOCHAT_BASE_DIR/engram_runs/engram_d12 --n-users 30
```